

Idempotency: Complete Guide & Interview Preparation

Table of Contents

1. [Core Concepts](#)
 2. [Idempotency Patterns](#)
 3. [HTTP Methods & Idempotency](#)
 4. [Implementation Strategies](#)
 5. [Real-World Scenarios](#)
 6. [Concurrent Writes in Microservices](#)
 7. [Storage Mechanisms](#)
 8. [Testing Idempotency](#)
 9. [Best Practices](#)
 10. [Interview Questions \(50\)](#)
-

1. Core Concepts

What is Idempotency?

Definition: An operation is idempotent if performing it multiple times has the same effect as performing it once.

Mathematical Definition:

$$f(f(x)) = f(x)$$

Example:

Setting a light switch to ON:

- First press: Light turns ON
- Second press: Light stays ON (same result)
- Nth press: Light stays ON (idempotent ✓)

vs

Toggling a light switch:

- First press: Light turns ON
- Second press: Light turns OFF (different result)
- Nth press: Alternates (NOT idempotent ✗)

Why Idempotency Matters

Scenario: Payment Processing

Without Idempotency:

User clicks "Pay \$100" → Network timeout → User retries → Charged \$200 ✗

With Idempotency:

User clicks "Pay \$100" (request_id: abc123)

- Network timeout
- User retries (same request_id: abc123)
- Server recognizes duplicate
- Returns original response
- Charged \$100 only ✓

Real-World Failure Scenarios

1. Double Charge Problem:

Timeline:

T1: Client sends payment request

T2: Server processes payment successfully

T3: Network fails before response reaches client

T4: Client times out, thinks it failed

T5: Client retries → DOUBLE CHARGE ✗

2. Duplicate Order Problem:

E-commerce Checkout:

- User clicks "Place Order" button
- Button doesn't disable fast enough
- User double-clicks
- Two orders created ✗

3. Webhook Retry Storm:

Payment Provider → Your Webhook:

- Sends webhook
- Your server is slow to respond
- Provider retries (thinks it failed)
- 10 retries = 10 duplicate notifications ✗

Idempotent vs Non-Idempotent Operations

Operation	Multiple Executions
GET /user/123	Same result every time ✓
PUT /user/123	Same state after 1st request ✓
DELETE /user/123	User deleted (idempotent) ✓
POST /users	Creates multiple users ✗
PATCH /user/inc-age	Age increases each time ✗

Transfer \$100	\$100 transferred each time X
----------------	-------------------------------

Visual Representation:

Idempotent Operation (SET):
Request 1: SET value = 10 → [value = 10]
Request 2: SET value = 10 → [value = 10] (same result)
Request 3: SET value = 10 → [value = 10] (same result)

Non-Idempotent Operation (INCREMENT):
Request 1: INCREMENT value → [value = 1]
Request 2: INCREMENT value → [value = 2] (different!)
Request 3: INCREMENT value → [value = 3] (different!)

2. Idempotency Patterns

Visual Overview: Four Core Patterns

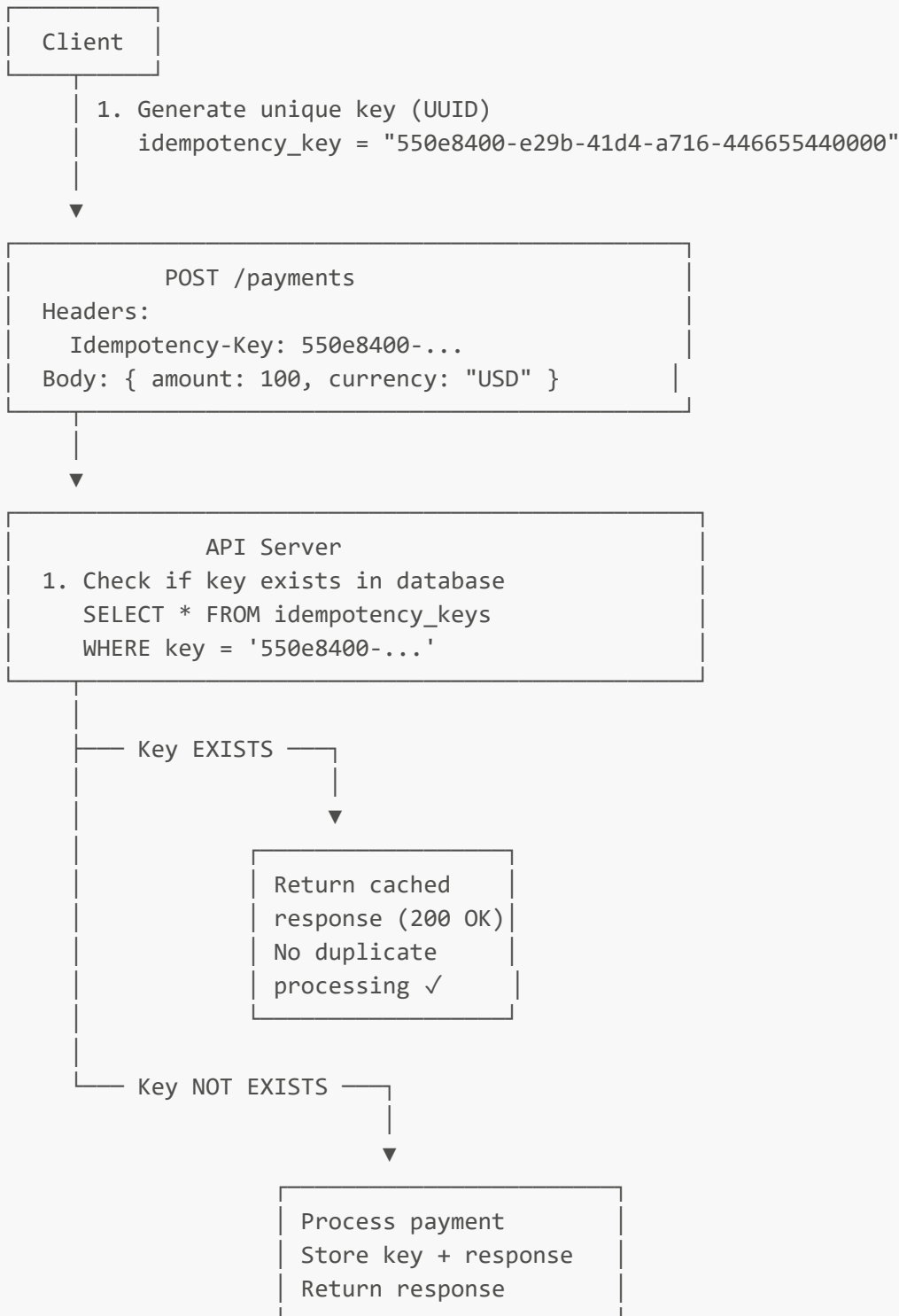
IDEMPOTENCY PATTERN SELECTION
1. DATABASE UNIQUE CONSTRAINTS Client → Server → Database (unique constraint) ✓ Simple, ACID guarantees X Database becomes bottleneck 2. IN-MEMORY TRACKING (Redis/Cache) Client → Server → Cache (check) → Database ✓ Fast, low latency X Cache invalidation complexity 3. DISTRIBUTED CACHE (Multi-Service) Client → Server → Redis → Database ✓ Scales across services X Cache consistency challenges 4. MESSAGE DEDUPLICATION (Event-Driven) API → Queue → Consumer (dedup window) ✓ Handles async operations X Time-window limitations

Reference: These patterns align with ByteByteGo's idempotent API design

Pattern 1: Idempotency Keys

How it works: Client generates unique key, server stores it to detect duplicates.

Flow Diagram:



Database Schema:

```
CREATE TABLE idempotency_keys (  
  key VARCHAR(255) PRIMARY KEY,  
  request_method VARCHAR(10),  
  request_path VARCHAR(500),  
  request_body JSONB,  
  response_status INT,  
  response_body JSONB,
```

```

        created_at TIMESTAMP,
        expires_at TIMESTAMP
    );

    CREATE INDEX idx_expires_at ON idempotency_keys(expires_at);

```

Implementation:

```

from fastapi import FastAPI, Header, HTTPException
import uuid
from datetime import datetime, timedelta
import hashlib
import json

app = FastAPI()

class IdempotencyStore:
    def __init__(self, db):
        self.db = db
        self.ttl = timedelta(hours=24) # Keys expire after 24 hours

    def check_and_store(self, key: str, request_data: dict) -> tuple[bool, dict]:
        """
        Check if idempotency key exists.
        Returns: (is_duplicate, stored_response)
        """
        # Check if key exists
        existing = self.db.execute(
            "SELECT response_body, response_status FROM idempotency_keys WHERE key"
            = %s",
            (key,)
        ).fetchone()

        if existing:
            # Duplicate request - return cached response
            return True, {
                'status': existing['response_status'],
                'body': existing['response_body']
            }

        # New request - store key (prevent concurrent duplicates)
        self.db.execute(
            """
            INSERT INTO idempotency_keys (key, request_body, created_at,
            expires_at)
            VALUES (%s, %s, %s, %s)
            ON CONFLICT (key) DO NOTHING
            """,
            (key, json.dumps(request_data), datetime.utcnow(), datetime.utcnow() +
            self.ttl)
        )

```

```

        return False, None

    def store_response(self, key: str, status: int, response: dict):
        """Store successful response"""
        self.db.execute(
            """
            UPDATE idempotency_keys
            SET response_status = %s, response_body = %s
            WHERE key = %s
            """,
            (status, json.dumps(response), key)
        )

    idempotency_store = IdempotencyStore(db)

@app.post("/payments")
async def create_payment(
    payment: PaymentRequest,
    idempotency_key: str = Header(None, alias="Idempotency-Key")
):
    # Require idempotency key
    if not idempotency_key:
        raise HTTPException(400, "Idempotency-Key header required")

    # Check for duplicate
    is_duplicate, cached_response = idempotency_store.check_and_store(
        idempotency_key,
        payment.dict()
    )

    if is_duplicate:
        # Return cached response
        return JSONResponse(
            content=cached_response['body'],
            status_code=cached_response['status']
        )

    try:
        # Process payment (only once!)
        result = await process_payment(payment)

        # Store response for future duplicates
        idempotency_store.store_response(
            idempotency_key,
            200,
            result
        )

        return result

    except Exception as e:
        # Don't cache errors (allow retry)
        raise

```

Pattern 2: Natural Idempotency (HTTP Methods)

```
GET /users/123      → Read (naturally idempotent)
PUT /users/123      → Replace entire resource (idempotent)
DELETE /users/123   → Delete resource (idempotent)
```

Why PUT is idempotent:

```
PUT /users/123 { "name": "John", "age": 30 }
→ First call: Sets name=John, age=30
→ Second call: Sets name=John, age=30 (same state!)
→ Nth call: Same state (idempotent ✓)
```

Why POST is NOT idempotent:

```
POST /users { "name": "John" }
→ First call: Creates user with id=1
→ Second call: Creates user with id=2 (different!)
→ NOT idempotent X
```

Pattern 3: Token-Based Idempotency (Stripe Style)

```
# Stripe-like implementation
class StripeStyleIdempotency:
    """
    Token lifecycle:
    1. Client generates token
    2. Server locks token during processing
    3. Server stores result with token
    4. Future requests with same token return stored result
    """

    def __init__(self, redis_client):
        self.redis = redis_client
        self.lock_ttl = 60 # Lock expires after 60 seconds
        self.result_ttl = 86400 # Results cached for 24 hours

    async def process_with_idempotency(
        self,
        idempotency_key: str,
        operation: callable
    ):
        """Process operation with idempotency guarantee"""

        # Step 1: Check if result already exists
        result_key = f"idem:result:{idempotency_key}"
        cached_result = await self.redis.get(result_key)

        if cached_result:
            return json.loads(cached_result)
```

```

# Step 2: Try to acquire lock
lock_key = f"idem:lock:{idempotency_key}"
lock_acquired = await self.redis.set(
    lock_key,
    "locked",
    nx=True, # Only set if not exists
    ex=self.lock_ttl
)

if not lock_acquired:
    # Another request is processing this key
    # Wait and check for result
    for _ in range(10): # Poll for 10 seconds
        await asyncio.sleep(1)
        cached_result = await self.redis.get(result_key)
        if cached_result:
            return json.loads(cached_result)

    raise HTTPException(409, "Request is being processed")

try:
    # Step 3: Execute operation (only once!)
    result = await operation()

    # Step 4: Store result
    await self.redis.setex(
        result_key,
        self.result_ttl,
        json.dumps(result)
    )

    return result

finally:
    # Step 5: Release lock
    await self.redis.delete(lock_key)

# Usage
stripe_idem = StripeStyleIdempotency(redis)

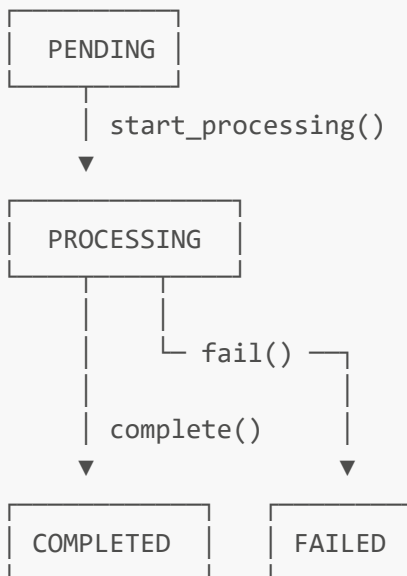
@app.post("/charges")
async def create_charge(
    charge: ChargeRequest,
    idempotency_key: str = Header(None)
):
    async def process_charge():
        # Actual payment processing
        return await payment_gateway.charge(charge)

    return await stripe_idem.process_with_idempotency(
        idempotency_key,
        process_charge
    )

```

Pattern 4: State Machine Pattern

Idempotent State Transitions:



Rules:

- PENDING → PROCESSING: OK (idempotent)
- PROCESSING → PROCESSING: OK (no change)
- COMPLETED → COMPLETED: OK (no change)
- COMPLETED → PROCESSING: ERROR (invalid transition)

Implementation:

```

from enum import Enum

class OrderStatus(Enum):
    PENDING = "pending"
    PROCESSING = "processing"
    COMPLETED = "completed"
    FAILED = "failed"

class OrderStateMachine:
    # Valid state transitions
    TRANSITIONS = {
        OrderStatus.PENDING: [OrderStatus.PROCESSING, OrderStatus.FAILED],
        OrderStatus.PROCESSING: [OrderStatus.COMPLETED, OrderStatus.FAILED],
        OrderStatus.COMPLETED: [], # Terminal state
        OrderStatus.FAILED: [OrderStatus.PENDING] # Allow retry
    }

    def transition(self, order_id: int, from_status: OrderStatus, to_status:
OrderStatus):
        """Idempotent state transition"""

```

```

query = """
    UPDATE orders
    SET status = %s, updated_at = NOW()
    WHERE id = %s AND status = %s
    RETURNING status
    """

result = db.execute(query, (to_status.value, order_id, from_status.value))

if result.rowcount == 0:
    # Check current status
    current = db.execute(
        "SELECT status FROM orders WHERE id = %s",
        (order_id,)
    ).fetchone()

    if current['status'] == to_status.value:
        # Already in target state - idempotent ✓
        return True
    else:
        # Invalid transition
        raise InvalidStateTransitionError(
            f"Cannot transition from {current['status']} to {to_status.value}"
        )

    return True

# Usage - Idempotent!
state_machine = OrderStateMachine()

# First call: PENDING → PROCESSING
state_machine.transition(order_id=123,
                        from_status=OrderStatus.PENDING,
                        to_status=OrderStatus.PROCESSING)

# Retry (network timeout): PENDING → PROCESSING
# Current status is already PROCESSING, so this is idempotent ✓
try:
    state_machine.transition(order_id=123,
                            from_status=OrderStatus.PENDING,
                            to_status=OrderStatus.PROCESSING)
except InvalidStateTransitionError:
    # Expected - order is already processing
    pass

```

Pattern 5: Compare-and-Swap (CAS)

Atomic compare-and-swap ensures idempotent updates:

Operation: Update balance if current value matches expected

Time	Thread A	Thread B
T1	Read balance=100	Read balance=100
T2	CAS(100 → 90) Success ✓	CAS(100 → 80) Fail X (value changed)
T3		Read balance=90 CAS(90 → 80) Success ✓

Final: balance=80 ✓ (correct result)

Implementation:

```
# PostgreSQL CAS
def withdraw_money(account_id: int, amount: int, expected_balance: int):
    """Idempotent withdrawal using CAS"""

    query = """
        UPDATE accounts
        SET balance = balance - %s,
            version = version + 1
        WHERE id = %s
            AND balance = %s -- Compare
            AND balance >= %s -- Ensure sufficient funds
        RETURNING balance, version
    """

    result = db.execute(
        query,
        (amount, account_id, expected_balance, amount)
    )

    if result.rowcount == 0:
        # CAS failed - balance changed or insufficient funds
        current = db.execute(
            "SELECT balance, version FROM accounts WHERE id = %s",
            (account_id,)
        ).fetchone()

        raise CASFailedError(
            f"Expected balance {expected_balance}, got {current['balance']}"
        )

    return result.fetchone()

# Redis CAS (WATCH + MULTI)
def redis_cas_update(key: str, expected_value: int, new_value: int):
    """Redis CAS using transactions"""

    with redis.pipeline() as pipe:
```

```
while True:
    try:
        pipe.watch(key)
        current_value = int(pipe.get(key))

        if current_value != expected_value:
            pipe.unwatch()
            raise CASFailedError(
                f"Expected {expected_value}, got {current_value}"
            )

        pipe.multi()
        pipe.set(key, new_value)
        pipe.execute()
        break

    except redis.WatchError:
        # Another client modified the key, retry
        continue
```

3. HTTP Methods & Idempotency

Standard HTTP Methods

Method	Idempotent?	Description
GET	✓ Yes	Read data, no side effects
HEAD	✓ Yes	Like GET but no body
OPTIONS	✓ Yes	Get allowed methods
PUT	✓ Yes	Replace entire resource
DELETE	✓ Yes	Delete resource
POST	✗ No	Create resource (generates new ID)
PATCH	✗ No*	Partial update (*depends on op)

* PATCH can be idempotent if operations are idempotent

Making POST Idempotent

```
# Non-Idempotent POST (default)
@app.post("/users")
def create_user(user: User):
    # Creates new user every time
    user_id = db.insert("INSERT INTO users ...", user)
    return {"id": user_id}

# Idempotent POST (with idempotency key)
```

```

@app.post("/users")
def create_user(
    user: User,
    idempotency_key: str = Header(None)
):
    if not idempotency_key:
        raise HTTPException(400, "Idempotency-Key required")

    # Check if already created
    existing = db.query(
        "SELECT * FROM users WHERE idempotency_key = %s",
        (idempotency_key,)
    )

    if existing:
        return existing # Return existing user

    # Create new user
    user.idempotency_key = idempotency_key
    user_id = db.insert("INSERT INTO users ...", user)
    return {"id": user_id}

# Schema
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255),
    email VARCHAR(255) UNIQUE,
    idempotency_key VARCHAR(255) UNIQUE, -- Ensures uniqueness
    created_at TIMESTAMP DEFAULT NOW()
);

```

PUT Idempotency Example

```

# PUT is naturally idempotent
@app.put("/users/{user_id}")
def update_user(user_id: int, user: User):
    """Replace entire user resource"""

    db.execute(
        """
        UPDATE users
        SET name = %s, email = %s, age = %s
        WHERE id = %s
        """,
        (user.name, user.email, user.age, user_id)
    )

    return user

# Multiple calls with same data = same result ✓
# PUT /users/123 {"name": "John", "age": 30}

```

```
# PUT /users/123 {"name": "John", "age": 30} # Same state
# PUT /users/123 {"name": "John", "age": 30} # Same state
```

PATCH - Idempotent vs Non-Idempotent

```
# Non-Idempotent PATCH (increment)
@app.patch("/users/{user_id}/increment-age")
def increment_age(user_id: int):
    db.execute("UPDATE users SET age = age + 1 WHERE id = %s", (user_id,))
    # Call 1: age = 31
    # Call 2: age = 32 (different!) X

# Idempotent PATCH (set value)
@app.patch("/users/{user_id}")
def patch_user(user_id: int, updates: dict):
    # Only SET operations
    if "age" in updates:
        db.execute("UPDATE users SET age = %s WHERE id = %s",
                    (updates["age"], user_id))
    # Call 1: age = 30
    # Call 2: age = 30 (same!) ✓
```

4. Implementation Strategies

Strategy 1: Database-Level (PostgreSQL)

```
-- Unique constraint prevents duplicates
CREATE TABLE payments (
    id SERIAL PRIMARY KEY,
    idempotency_key VARCHAR(255) UNIQUE NOT NULL,
    user_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    status VARCHAR(50) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT NOW()
);

-- Attempt to insert
INSERT INTO payments (idempotency_key, user_id, amount)
VALUES ('key-123', 42, 100.00)
ON CONFLICT (idempotency_key) DO NOTHING
RETURNING *;

-- If no rows returned, payment already exists
```

Implementation:

```
def create_payment_idempotent(idempotency_key: str, user_id: int, amount: float):
    """Database-level idempotency"""

    result = db.execute(
        """
        INSERT INTO payments (idempotency_key, user_id, amount)
        VALUES (%s, %s, %s)
        ON CONFLICT (idempotency_key) DO UPDATE
            SET idempotency_key = EXCLUDED.idempotency_key -- No-op update
        RETURNING id, status, created_at,
            (xmax = 0) AS inserted -- True if new insert
        """,
        (idempotency_key, user_id, amount)
    ).fetchone()

    if result['inserted']:
        # New payment created
        return {'id': result['id'], 'status': 'created'}
    else:
        # Payment already exists
        return {'id': result['id'], 'status': 'duplicate'}
```

Strategy 2: Redis-Based

```
import redis
import json
from datetime import timedelta

class RedisIdempotency:
    def __init__(self, redis_client, ttl=timedelta(hours=24)):
        self.redis = redis_client
        self.ttl = int(ttl.total_seconds())

    def execute_once(self, key: str, operation: callable, *args, **kwargs):
        """Execute operation only once per key"""

        # Check if already executed
        result = self.redis.get(f"idem:{key}")
        if result:
            return json.loads(result)

        # Use Redis lock to prevent concurrent execution
        lock = self.redis.lock(f"lock:{key}", timeout=30)

        if lock.acquire(blocking=True, blocking_timeout=10):
            try:
                # Double-check after acquiring lock
                result = self.redis.get(f"idem:{key}")
                if result:
                    return json.loads(result)
```

```

        # Execute operation
        result = operation(*args, **kwargs)

        # Store result
        self.redis.setex(
            f"idem:{key}",
            self.ttl,
            json.dumps(result)
        )

        return result
    finally:
        lock.release()
    else:
        raise TimeoutError("Could not acquire lock")

# Usage
redis_idem = RedisIdempotency(redis.Redis())

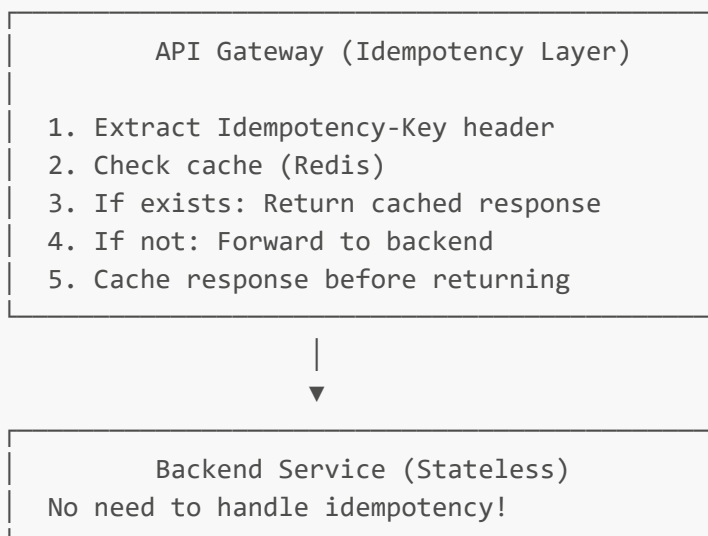
@app.post("/orders")
async def create_order(order: Order, idempotency_key: str = Header(None)):
    def process_order():
        # Create order in database
        order_id = db.insert_order(order)
        return {"order_id": order_id, "status": "created"}

    return redis_idem.execute_once(idempotency_key, process_order)

```

Strategy 3: API Gateway Level

API Gateway (Kong/AWS API Gateway):



Kong Plugin Example:

```

-- Kong idempotency plugin
local IdempotencyHandler = {
    PRIORITY = 1000,
    VERSION = "1.0.0"
}

function IdempotencyHandler:access(conf)
    local key = kong.request.get_header("Idempotency-Key")

    if not key then
        return -- No idempotency key, proceed normally
    end

    -- Check Redis cache
    local redis = require "resty.redis"
    local red = redis:new()
    red:connect("127.0.0.1", 6379)

    local cached_response = red:get("idem:" .. key)

    if cached_response and cached_response ~= ngx.null then
        -- Return cached response
        local response = cJSON.decode(cached_response)
        return kong.response.exit(response.status, response.body)
    end

    -- Mark for caching in response phase
    kong.ctx.plugin.idempotency_key = key
end

function IdempotencyHandler:response(conf)
    local key = kong.ctx.plugin.idempotency_key

    if key then
        -- Cache the response
        local status = ngx.status
        local body = kong.response.get_raw_body()

        local response_data = cJSON.encode({
            status = status,
            body = body
        })

        red:setex("idem:" .. key, 86400, response_data) -- 24h TTL
    end
end

return IdempotencyHandler

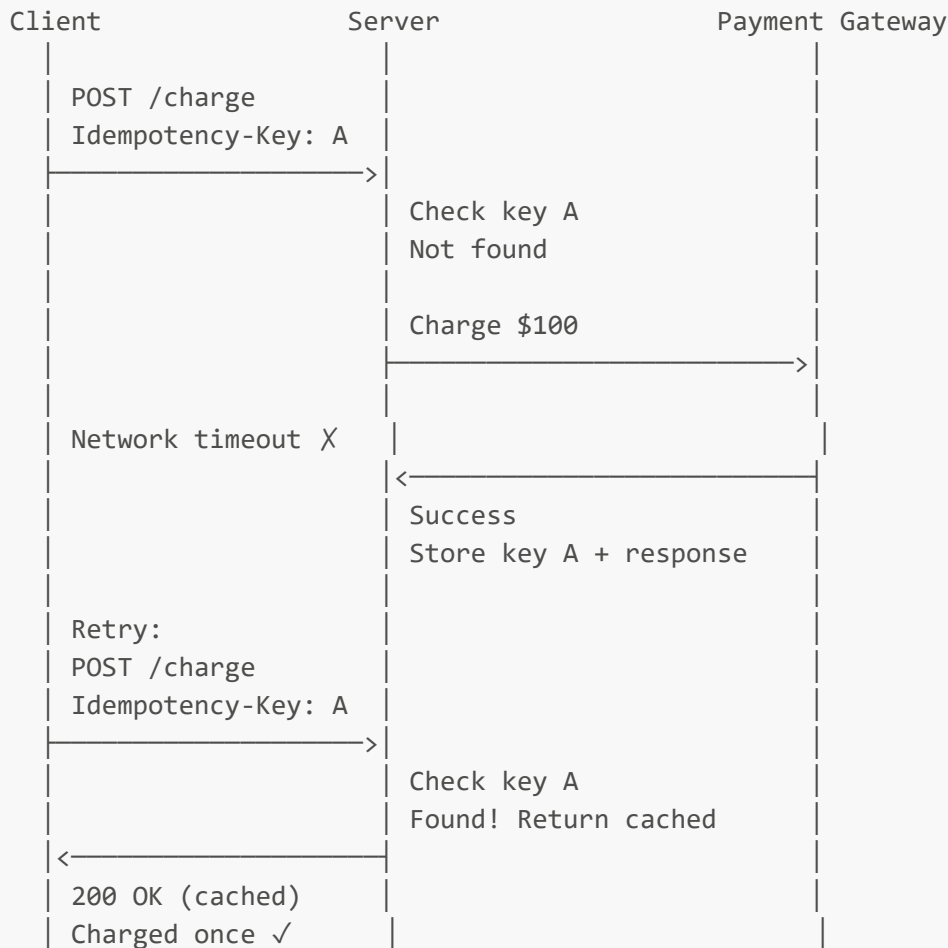
```

5. Real-World Scenarios

Scenario 1: Payment Processing

Problem: Prevent double charges

Flow with Idempotency:



Implementation:

```

@app.post("/payments")
async def process_payment(
    payment: PaymentRequest,
    idempotency_key: str = Header(...)
):
    # Check if payment already processed
    existing_payment = await db.fetchone(
        "SELECT * FROM payments WHERE idempotency_key = %s",
        (idempotency_key,)
    )

    if existing_payment:
        # Return existing payment (idempotent)
        return existing_payment

    try:

```

```

# Process payment with external gateway
gateway_response = await payment_gateway.charge(
    amount=payment.amount,
    currency=payment.currency,
    source=payment.card_token
)

# Store payment record
payment_record = await db.execute(
    """
    INSERT INTO payments (
        idempotency_key,
        amount,
        currency,
        gateway_transaction_id,
        status
    ) VALUES (%s, %s, %s, %s, %s)
    RETURNING *
    """,
    (
        idempotency_key,
        payment.amount,
        payment.currency,
        gateway_response.transaction_id,
        'completed'
    )
)

return payment_record

except PaymentGatewayError as e:
    # Don't cache failures - allow retry
    raise HTTPException(500, str(e))

```

Scenario 2: Order Creation (Button Double-Click)

Problem: User double-clicks "Place Order" button

Solution: Client-side generated idempotency key

Frontend:

```

async function placeOrder(orderData) {
    // Generate idempotency key on client
    const idempotencyKey = `order-${Date.now()}-${Math.random()}`;

    // Disable button
    const button = document.getElementById('place-order-btn');
    button.disabled = true;

```

```

    try {
      const response = await fetch('/api/orders', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Idempotency-Key': idempotencyKey
        },
        body: JSON.stringify(orderData)
      });

      return await response.json();
    } finally {
      // Re-enable button after response
      button.disabled = false;
    }
  }
}

```

Backend:

```

@app.post("/orders")
async def create_order(
    order: OrderRequest,
    idempotency_key: str = Header(...)
):
    # Try to create order
    try:
        order_id = await db.execute(
            """
            INSERT INTO orders (
                idempotency_key,
                user_id,
                items,
                total,
                status
            ) VALUES (%s, %s, %s, %s, 'pending')
            RETURNING id
            """,
            (idempotency_key, order.user_id, order.items, order.total)
        )

        return {"order_id": order_id, "status": "created"}

    except UniqueViolationError:
        # Idempotency key already exists
        existing_order = await db.fetchone(
            "SELECT id, status FROM orders WHERE idempotency_key = %s",
            (idempotency_key,)
        )
        return {"order_id": existing_order['id'], "status": "duplicate"}

```

Scenario 3: Webhook Processing

Problem: Payment provider retries webhook 10 times

Webhook Flow:

Provider sends: payment.success event

- Your server is slow (> 5s response)
- Provider times out, thinks it failed
- Provider retries after 1 minute
- Your server processes duplicate event X

Solution: Idempotent webhook handler

```
@app.post("/webhooks/payments")
async def handle_payment_webhook(webhook: WebhookPayload, request: Request):
    # Extract webhook ID (provider-generated)
    webhook_id = webhook.id # e.g., "evt_1a2b3c4d"

    # Check if already processed
    processed = await redis.get(f"webhook:processed:{webhook_id}")
    if processed:
        # Already processed - return success to stop retries
        return {"status": "ok", "message": "already_processed"}

    try:
        # Process webhook
        if webhook.type == "payment.success":
            await mark_payment_as_paid(webhook.payment_id)
        elif webhook.type == "payment.failed":
            await mark_payment_as_failed(webhook.payment_id)

        # Mark as processed (TTL: 7 days)
        await redis.setex(f"webhook:processed:{webhook_id}", 604800, "1")

        return {"status": "ok"}

    except Exception as e:
        # Don't mark as processed if failed - allow retry
        logger.error(f"Webhook processing failed: {e}")
        raise HTTPException(500, "Processing failed")
```

Scenario 4: Email Sending

Problem: Sending the same email multiple times

Solution: Idempotent email sender

```

class IdempotentEmailSender:
    def __init__(self, redis_client, email_service):
        self.redis = redis_client
        self.email_service = email_service
        self.ttl = 86400 # 24 hours

    async def send_email(self, to: str, subject: str, body: str, idempotency_key:
str):
        """Send email only once per idempotency key"""

        # Check if already sent
        sent_key = f"email:sent:{idempotency_key}"
        if await self.redis.exists(sent_key):
            logger.info(f"Email already sent: {idempotency_key}")
            return {"status": "already_sent", "idempotency_key": idempotency_key}

        try:
            # Send email
            message_id = await self.email_service.send(
                to=to,
                subject=subject,
                body=body
            )

            # Mark as sent
            await self.redis.setex(sent_key, self.ttl, message_id)

            return {
                "status": "sent",
                "message_id": message_id,
                "idempotency_key": idempotency_key
            }

        except EmailServiceError as e:
            # Don't mark as sent if failed
            logger.error(f"Email send failed: {e}")
            raise

# Usage
email_sender = IdempotentEmailSender(redis, email_service)

# Order confirmation email
await email_sender.send_email(
    to=user.email,
    subject="Order Confirmation",
    body=f"Your order #{order.id} has been confirmed",
    idempotency_key=f"order-confirmation-{order.id}"
)

# Even if called multiple times, email sent only once ✓

```

6. Concurrent Writes in Microservices

The Problem: Race Conditions

Scenario: Two services updating inventory simultaneously

Time	Service A (Order)	Service B (Return)	Database
T1	Read stock = 100	Read stock = 100	stock = 100
T2	Calculate: 100 - 5 = 95	Calculate: 100 + 2 = 102	
T3	Write stock = 95	Write stock = 102	stock = 102
T4			✗ WRONG! (Should be 97)

Lost Update Problem: Last write wins, earlier update lost

Solution 1: Pessimistic Locking (SELECT FOR UPDATE)

Concept: Lock the row before reading, preventing concurrent modifications.

```
-- Transaction 1 (Service A)
BEGIN;
SELECT stock FROM inventory WHERE product_id = 123 FOR UPDATE;
-- Row is now locked, Service B must wait
UPDATE inventory SET stock = stock - 5 WHERE product_id = 123;
COMMIT;
-- Lock released

-- Transaction 2 (Service B) - waits until Transaction 1 commits
BEGIN;
SELECT stock FROM inventory WHERE product_id = 123 FOR UPDATE;
-- Can now acquire lock
UPDATE inventory SET stock = stock + 2 WHERE product_id = 123;
COMMIT;
```

Implementation:

```
from sqlalchemy.orm import Session
from sqlalchemy import select

def decrease_stock_pessimistic(db: Session, product_id: int, quantity: int):
    """Pessimistic locking with SELECT FOR UPDATE"""

    # Start transaction
    with db.begin():
```

```

# Lock the row
result = db.execute(
    select(Inventory)
    .where(Inventory.product_id == product_id)
    .with_for_update() # SELECT FOR UPDATE
)
inventory = result.scalar_one()

# Check stock
if inventory.stock < quantity:
    raise InsufficientStockError()

# Update stock (while holding lock)
inventory.stock -= quantity
db.commit() # Lock released here

return inventory.stock

# Usage - Sequential execution guaranteed
# Service A and B cannot run simultaneously
stock = decrease_stock_pessimistic(db, product_id=123, quantity=5)

```

Pros & Cons:

- ✓ Pros:
 - Guaranteed consistency
 - Simple to understand
 - No lost updates
- ✗ Cons:
 - Blocks concurrent requests (slow)
 - Can cause deadlocks
 - Poor performance under high contention
 - Doesn't scale well

Deadlock Example:

```

-- Transaction 1
BEGIN;
SELECT * FROM orders WHERE id = 1 FOR UPDATE;      -- Locks order 1
SELECT * FROM inventory WHERE id = 100 FOR UPDATE; -- Waits for inventory

-- Transaction 2 (concurrent)
BEGIN;
SELECT * FROM inventory WHERE id = 100 FOR UPDATE; -- Locks inventory 100
SELECT * FROM orders WHERE id = 1 FOR UPDATE;      -- Waits for order 1

-- DEADLOCK! Each transaction waiting for the other
-- Database detects and aborts one transaction

```

Solution 2: Optimistic Locking (Version Numbers)

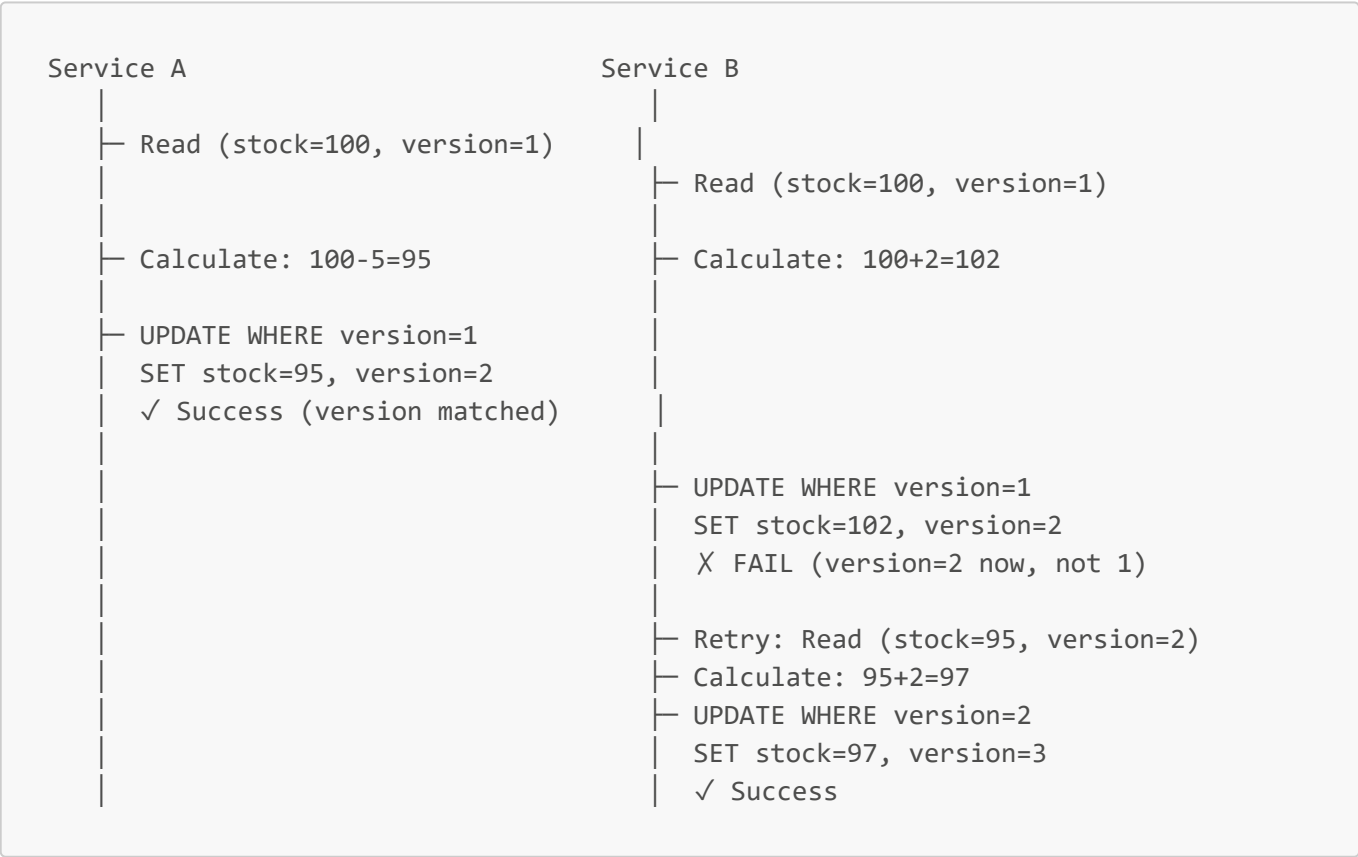
Concept: Check if data changed since read, abort if it did.

```
-- Add version column
CREATE TABLE inventory (
  product_id INT PRIMARY KEY,
  stock INT NOT NULL,
  version INT NOT NULL DEFAULT 1
);

-- Optimistic update
UPDATE inventory
SET stock = stock - 5,
    version = version + 1
WHERE product_id = 123
  AND version = 10 -- Expected version
RETURNING stock, version;

-- If no rows updated: version mismatch, retry
```

Flow Diagram:



Implementation:

```

from typing import Optional

class OptimisticLockError(Exception):
    pass

def decrease_stock_optimistic(
    db: Session,
    product_id: int,
    quantity: int,
    max_retries: int = 3
) -> int:
    """Optimistic locking with automatic retry"""

    for attempt in range(max_retries):
        try:
            # Read current state (no lock)
            inventory = db.query(Inventory).filter(
                Inventory.product_id == product_id
            ).first()

            if not inventory:
                raise NotFoundError()

            current_version = inventory.version
            current_stock = inventory.stock

            # Check sufficient stock
            if current_stock < quantity:
                raise InsufficientStockError()

            # Try to update with version check
            rows_updated = db.execute(
                update(Inventory)
                .where(Inventory.product_id == product_id)
                .where(Inventory.version == current_version) # Optimistic lock
                .values(
                    stock=Inventory.stock - quantity,
                    version=Inventory.version + 1
                )
            ).rowcount

            if rows_updated == 0:
                # Version mismatch - data changed by another transaction
                db.rollback()
                if attempt < max_retries - 1:
                    time.sleep(0.01 * (2 ** attempt)) # Exponential backoff
                    continue # Retry
                else:
                    raise OptimisticLockError("Max retries exceeded")

            db.commit()
            return current_stock - quantity

```

```

        except Exception as e:
            db.rollback()
            raise

        raise OptimisticLockError("Failed to update after retries")

# Usage - Concurrent requests handled gracefully
stock = decrease_stock_optimistic(db, product_id=123, quantity=5)

```

Pros & Cons:

- ✓ Pros:
 - No blocking (better performance)
 - Scales well with low contention
 - No deadlocks
 - Simple implementation
- ✗ Cons:
 - Requires retry logic
 - Can fail under high contention
 - Application must handle conflicts
 - Not suitable for high-conflict scenarios

Solution 3: Distributed Locks

3A. Redis Simple Lock (SETNX)

Redis SET command with NX (Not eXists) and EX (EXpiration):

```
SET lock:inventory:123 "owner-id" NX EX 10
```

NX: Only set if key doesn't exist

EX 10: Expire after 10 seconds (prevent deadlock)

Implementation:

```

import redis
import uuid
import time

class RedisLock:
    def __init__(self, redis_client, key: str, timeout: int = 10):
        self.redis = redis_client
        self.key = f"lock:{key}"
        self.timeout = timeout
        self.owner = str(uuid.uuid4()) # Unique owner ID

```

```

def acquire(self, blocking: bool = True, timeout: float = 10) -> bool:
    """Acquire distributed lock"""

    start_time = time.time()

    while True:
        # Try to acquire lock
        acquired = self.redis.set(
            self.key,
            self.owner,
            nx=True, # Only if not exists
            ex=self.timeout # Expire after timeout seconds
        )

        if acquired:
            return True

        if not blocking:
            return False

        # Check timeout
        if time.time() - start_time > timeout:
            return False

        # Wait before retry
        time.sleep(0.01)

def release(self):
    """Release lock (only if we own it)"""

    # Lua script for atomic check-and-delete
    lua_script = """
    if redis.call("get", KEYS[1]) == ARGV[1] then
        return redis.call("del", KEYS[1])
    else
        return 0
    end
    """

    result = self.redis.eval(lua_script, 1, self.key, self.owner)
    return bool(result)

def __enter__(self):
    if not self.acquire():
        raise TimeoutError("Could not acquire lock")
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    self.release()

```

Usage

```
redis_client = redis.Redis(host='localhost', port=6379)
```

```
def update_inventory_with_lock(product_id: int, quantity: int):
```

```

"""Use distributed lock to prevent race conditions"""

with RedisLock(redis_client, f"inventory:{product_id}"):
    # Critical section - only one process can execute this
    inventory = db.query(Inventory).filter(
        Inventory.product_id == product_id
    ).first()

    if inventory.stock < quantity:
        raise InsufficientStockError()

    inventory.stock -= quantity
    db.commit()

# Multiple services can call this safely
update_inventory_with_lock(product_id=123, quantity=5)

```

Edge Cases:

```

# Problem: Lock holder crashes before releasing
# Solution: Automatic expiration

# Problem: Operation takes longer than lock timeout
# Solution: Lock renewal/heartbeat

class RenewableRedisLock(RedisLock):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.renewal_thread = None
        self.should_renew = False

    def acquire(self, *args, **kwargs):
        acquired = super().acquire(*args, **kwargs)

        if acquired:
            # Start renewal thread
            self.should_renew = True
            self.renewal_thread = threading.Thread(target=self._renew_lock)
            self.renewal_thread.daemon = True
            self.renewal_thread.start()

        return acquired

    def _renew_lock(self):
        """Periodically renew lock before expiration"""
        while self.should_renew:
            time.sleep(self.timeout / 2) # Renew at half timeout

            # Extend expiration
            lua_script = """
            if redis.call("get", KEYS[1]) == ARGV[1] then
                return redis.call("expire", KEYS[1], ARGV[2])
            end
            """

```

```

        else
            return 0
        end
    """
    self.redis.eval(lua_script, 1, self.key, self.owner, self.timeout)

def release(self):
    self.should_renew = False
    return super().release()

```

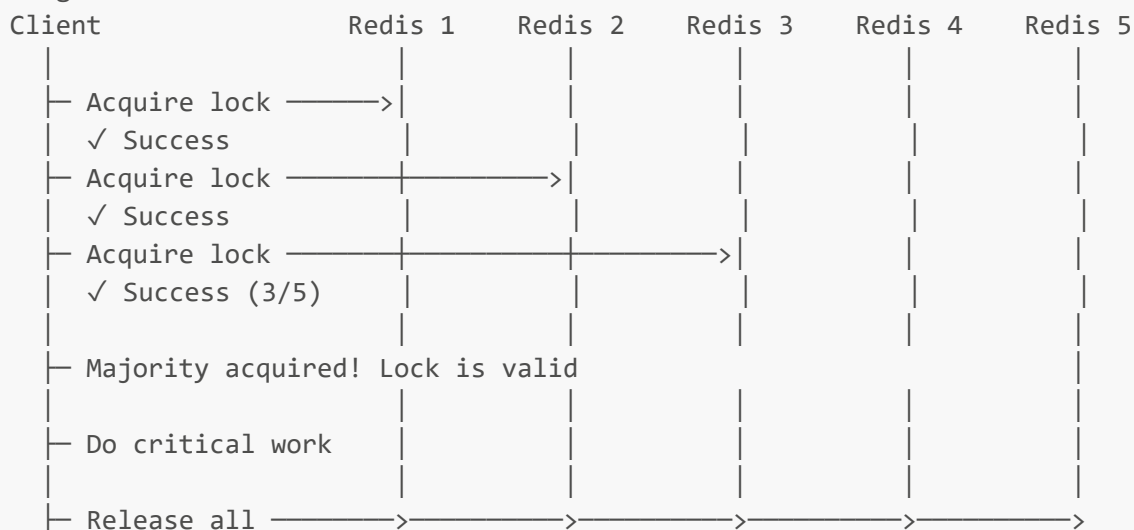
3B. Redlock Algorithm (Multiple Redis Instances)

Concept: Acquire locks on majority of Redis instances for safety.

Redlock with 5 Redis instances:

- Step 1: Get current time (milliseconds)
- Step 2: Try to acquire lock on all 5 instances sequentially
 - Use same key and value
 - Use small timeout (< lock TTL)
- Step 3: Check if acquired lock on majority (3 out of 5)
- Step 4: If yes: Calculate remaining lock validity time
If no: Release all locks and retry
- Step 5: Do work
- Step 6: Release all locks

Diagram:



Implementation:

```

import time
import random

class Redlock:
    """Redlock algorithm for distributed locking"""

```

```

def __init__(self, redis_instances: list, ttl_ms: int = 10000):
    self.instances = redis_instances # List of Redis clients
    self.ttl_ms = ttl_ms
    self.retry_count = 3
    self.retry_delay_ms = 200
    self.clock_drift_factor = 0.01 # 1%

def acquire(self, resource: str, val: str) -> bool:
    """Try to acquire lock on majority of instances"""

    for _ in range(self.retry_count):
        # Try to acquire on all instances
        start_time = int(time.time() * 1000)
        acquired_count = 0

        for redis_client in self.instances:
            try:
                # Set with NX and PX
                if redis_client.set(resource, val, nx=True, px=self.ttl_ms):
                    acquired_count += 1
            except Exception as e:
                # If instance is down, continue
                pass

        # Calculate elapsed time and drift
        elapsed_ms = int(time.time() * 1000) - start_time
        drift = int(self.ttl_ms * self.clock_drift_factor) + 2
        validity_time = self.ttl_ms - elapsed_ms - drift

        # Check if acquired majority
        if acquired_count >= (len(self.instances) // 2 + 1) and validity_time
> 0:
            return True

        # Failed to acquire majority - release all locks
        self.release(resource, val)

        # Wait before retry
        time.sleep(random.uniform(0, self.retry_delay_ms) / 1000)

    return False

def release(self, resource: str, val: str):
    """Release lock on all instances"""

    lua_script = """
    if redis.call("get", KEYS[1]) == ARGV[1] then
        return redis.call("del", KEYS[1])
    else
        return 0
    end
    """

```

```

        for redis_client in self.instances:
            try:
                redis_client.eval(lua_script, 1, resource, val)
            except:
                pass # Best effort

# Usage with 5 Redis instances
redis_instances = [
    redis.Redis(host='redis1.example.com', port=6379),
    redis.Redis(host='redis2.example.com', port=6379),
    redis.Redis(host='redis3.example.com', port=6379),
    redis.Redis(host='redis4.example.com', port=6379),
    redis.Redis(host='redis5.example.com', port=6379),
]

redlock = Redlock(redis_instances, ttl_ms=10000)

resource = "inventory:123"
lock_value = str(uuid.uuid4())

if redlock.acquire(resource, lock_value):
    try:
        # Critical section
        update_inventory(product_id=123, quantity=5)
    finally:
        redlock.release(resource, lock_value)
else:
    raise LockAcquisitionError("Could not acquire lock")

```

Controversy: Redlock Debate

Martin Kleppmann's Critique:

- ✗ Clock drift can break safety
- ✗ Process pauses (GC) can cause issues
- ✗ Not safe with async replication
- ✗ Complexity not worth it

Salvatore Sanfilippo's Defense:

- ✓ Good enough for most use cases
- ✓ Clock drift bounded in practice
- ✓ Better than single Redis
- ✓ Simple alternative to ZooKeeper

Recommendation:

- Use Redlock for: Non-critical operations, short locks
- Don't use for: Financial transactions, inventory
- Use ZooKeeper/etcd for: Critical operations

3C. ZooKeeper Distributed Lock

Concept: Use ephemeral sequential nodes for lock queue.

ZooKeeper Lock Algorithm:

1. Create ephemeral sequential node: /locks/resource-0000000001
2. Get all children of /locks
3. If your node is smallest: You have the lock
4. If not: Watch the node before yours
5. When that node deleted: Check again if you're smallest
6. Do work
7. Delete your node (releases lock)

Diagram:

```
/locks/
├─ inventory-0000000001 ← Client A (has lock)
├─ inventory-0000000002 ← Client B (waiting, watches #1)
└─ inventory-0000000003 ← Client C (waiting, watches #2)
```

Client A finishes → Deletes node #1

Client B wakes up → Now has lock

Implementation (Python with Kazoo):

```
from kazoo.client import KazooClient
from kazoo.recipe.lock import Lock

class ZooKeeperLock:
    def __init__(self, hosts: str, path: str):
        self.zk = KazooClient(hosts=hosts)
        self.zk.start()
        self.lock = Lock(self.zk, path)

    def __enter__(self):
        self.lock.acquire()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.lock.release()

    def close(self):
        self.zk.stop()

# Usage
zk_lock = ZooKeeperLock(
    hosts='zk1:2181,zk2:2181,zk3:2181',
    path='/locks/inventory-123'
)

with zk_lock:
```

```
# Critical section - only one client can execute
update_inventory(product_id=123, quantity=5)
```

Java Implementation (Apache Curator):

```
import org.apache.curator.framework.CuratorFramework;
import org.apache.curator.framework.CuratorFrameworkFactory;
import org.apache.curator.framework.recipes.locks.InterProcessMutex;
import org.apache.curator.retry.ExponentialBackoffRetry;

public class ZooKeeperDistributedLock {
    private CuratorFramework client;
    private InterProcessMutex lock;

    public ZooKeeperDistributedLock(String zkConnection, String lockPath) {
        client = CuratorFrameworkFactory.newClient(
            zkConnection,
            new ExponentialBackoffRetry(1000, 3)
        );
        client.start();
        lock = new InterProcessMutex(client, lockPath);
    }

    public void executeWithLock(Runnable task) throws Exception {
        if (lock.acquire(10, TimeUnit.SECONDS)) {
            try {
                task.run();
            } finally {
                lock.release();
            }
        } else {
            throw new TimeoutException("Could not acquire lock");
        }
    }
}

// Usage
ZooKeeperDistributedLock lock = new ZooKeeperDistributedLock(
    "zk1:2181,zk2:2181,zk3:2181",
    "/locks/inventory-123"
);

lock.executeWithLock(() -> {
    updateInventory(productId, quantity);
});
```

3D. etcd Distributed Lock

Concept: Use lease-based locking with automatic expiration.

etcd Lock Flow:

1. Create lease with TTL
2. Put key with lease
3. If successful: Lock acquired
4. Keep lease alive (heartbeat)
5. Do work
6. Delete key or let lease expire

Implementation (Go):

```
package main

import (
    "context"
    "time"

    clientv3 "go.etcd.io/etcd/client/v3"
    "go.etcd.io/etcd/client/v3/concurrency"
)

type EtcdLock struct {
    client *clientv3.Client
    session *concurrency.Session
    mutex *concurrency.Mutex
}

func NewEtcdLock(endpoints []string, key string) (*EtcdLock, error) {
    // Create etcd client
    client, err := clientv3.New(clientv3.Config{
        Endpoints: endpoints,
        DialTimeout: 5 * time.Second,
    })
    if err != nil {
        return nil, err
    }

    // Create session with TTL
    session, err := concurrency.NewSession(client, concurrency.WithTTL(10))
    if err != nil {
        return nil, err
    }

    // Create mutex
    mutex := concurrency.NewMutex(session, "/locks/"+key)

    return &EtcdLock{
        client: client,
        session: session,
        mutex: mutex,
    }, nil
}
```

```

}

func (l *EtdcdLock) Lock(ctx context.Context) error {
    return l.mutex.Lock(ctx)
}

func (l *EtdcdLock) Unlock(ctx context.Context) error {
    return l.mutex.Unlock(ctx)
}

func (l *EtdcdLock) Close() error {
    l.session.Close()
    return l.client.Close()
}

// Usage
func main() {
    lock, err := NewEtdcdLock(
        []string{"etcd1:2379", "etcd2:2379", "etcd3:2379"},
        "inventory-123",
    )
    if err != nil {
        panic(err)
    }
    defer lock.Close()

    ctx := context.Background()

    // Acquire lock
    if err := lock.Lock(ctx); err != nil {
        panic(err)
    }
    defer lock.Unlock(ctx)

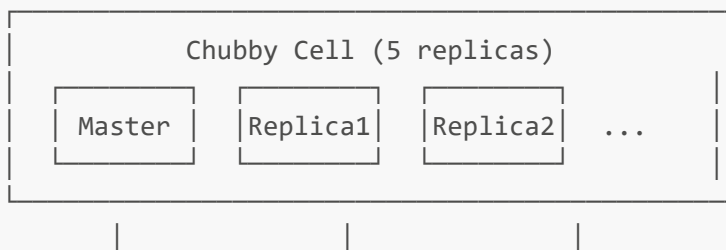
    // Critical section
    updateInventory(productId, quantity)
}

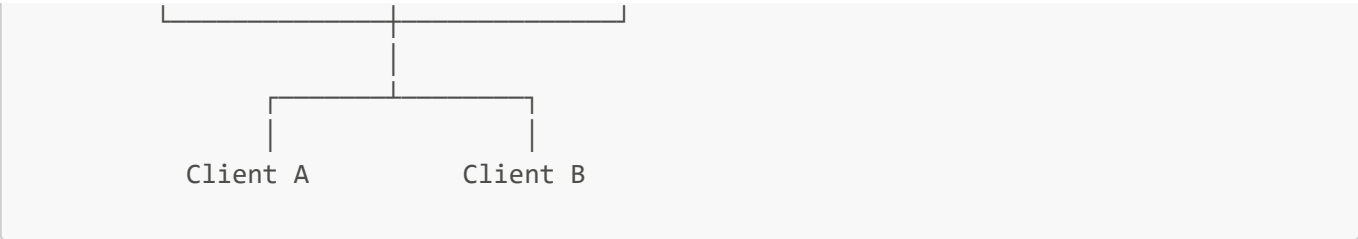
```

3E. Chubby (Google - Conceptual)

Concept: Coarse-grained distributed lock service.

Chubby Architecture:





Key Features:

- File-based lock metaphor
- Advisory locks (not mandatory)
- Session-based leases
- Event notification
- Used by: Bigtable, GFS, Megastore

Conceptual API:

```
# Conceptual Chubby API (not real code)

# Open lock file
handle = chubby.Open("/ls/cell/app/lock", CREATE | WRITE)

# Acquire lock
handle.Acquire(WRITE_LOCK, timeout=10)

try:
    # Critical section
    update_inventory()
finally:
    # Release lock
    handle.Release()
    handle.Close()
```

Why Google built Chubby:

- Needed reliable distributed coordination
- ZooKeeper didn't exist yet
- Optimized for coarse-grained locks (minutes/hours)
- Not open-source (use ZooKeeper/etcd instead)

Comparison Matrix: Distributed Locking Solutions

Feature Chubby	Redis Simple	Redlock	ZooKeeper	etcd	
Consistency	Single point	Majority	Strong	Strong	

Strong (Paxos)		of failure	quorum	(Paxos/Zab)	(Raft)	
Complexity	Low	Medium	High	Medium		
High						
Performance	Very High	High	Medium	High		
Medium						
Fault Tolerance	Low	Medium	High	High		
Very High						
Lock Fairness	No	No	Yes (FIFO)	Yes	Yes	
Setup	Very Easy	Medium	Complex	Medium	N/A	
Min Instances	1	5 (odd)	3 (odd)	3 (odd)	5	
Use Case	Low-critical	Medium	Critical	Critical		
Google-scale	operations	critical	operations	Kubernetes		
internal						
Best For	Caching,	Multi-DC	Config mgmt,	K8s, service		
Bigtable,	rate	coordination	leader	discovery	GFS	
	limiting		election			
Latency	< 1ms	5-10ms	10-50ms	10-30ms	10-	
50ms						
Split Brain	Possible	Unlikely	No	No	No	
Safety						
Auto Expiry	Yes	Yes	Yes	Yes (lease)	Yes	



Solution 4: DynamoDB Conditional Writes

Concept: Use condition expressions to ensure atomic updates.

```

import boto3
from botocore.exceptions import ClientError

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('inventory')

def decrease_stock_dynamodb(product_id: str, quantity: int):
    """Atomic conditional write in DynamoDB"""

    try:
        response = table.update_item(
            Key={'product_id': product_id},
            UpdateExpression='SET stock = stock - :qty, version = version + :inc',
            ConditionExpression='stock >= :qty AND version = :expected_version',
            ExpressionAttributeValues={
                ':qty': quantity,
                ':inc': 1,
                ':expected_version': current_version # Read first
            },
            ReturnValues='ALL_NEW'
        )
        return response['Attributes']

    except ClientError as e:
        if e.response['Error']['Code'] == 'ConditionalCheckFailedException':
            # Condition failed - either insufficient stock or version mismatch
            raise ConflictError("Stock update failed - retry")
        raise

# Idempotency with DynamoDB
def process_payment_idempotent(payment_id: str, idempotency_key: str):
    """Idempotent payment processing with DynamoDB"""

    try:
        # Try to insert payment record
        table.put_item(
            Item={
                'payment_id': payment_id,
                'idempotency_key': idempotency_key,
            }
        )
    except ClientError as e:
        if e.response['Error']['Code'] == 'ConditionalCheckFailedException':
            # Condition failed - either insufficient stock or version mismatch
            raise ConflictError("Payment update failed - retry")
        raise

```

```

        'status': 'processing',
        'created_at': datetime.utcnow().isoformat()
    },
    ConditionExpression='attribute_not_exists(idempotency_key)'
)

# Process payment
result = charge_payment(payment_id)

# Update status
table.update_item(
    Key={'payment_id': payment_id},
    UpdateExpression='SET #status = :status',
    ExpressionAttributeNames={'#status': 'status'},
    ExpressionAttributeValues={':status': 'completed'}
)

return result

except ClientError as e:
    if e.response['Error']['Code'] == 'ConditionalCheckFailedException':
        # Idempotency key already exists - return existing payment
        response = table.get_item(
            Key={'payment_id': payment_id}
        )
        return response['Item']
    raise

```

DynamoDB Transaction Example:

```

def transfer_money_atomic(from_account: str, to_account: str, amount: int):
    """Atomic money transfer with DynamoDB transactions"""

    try:
        dynamodb_client = boto3.client('dynamodb')

        response = dynamodb_client.transact_write_items(
            TransactItems=[
                {
                    'Update': {
                        'TableName': 'accounts',
                        'Key': {'account_id': {'S': from_account}},
                        'UpdateExpression': 'SET balance = balance - :amount',
                        'ConditionExpression': 'balance >= :amount',
                        'ExpressionAttributeValues': {
                            ':amount': {'N': str(amount)}
                        }
                    }
                },
                {
                    'Update': {
                        'TableName': 'accounts',

```

```

        'Key': {'account_id': {'S': to_account}},
        'UpdateExpression': 'SET balance = balance + :amount',
        'ExpressionAttributeValues': {
            ':amount': {'N': str(amount)}
        }
    }
]
)

return {'status': 'success', 'amount': amount}

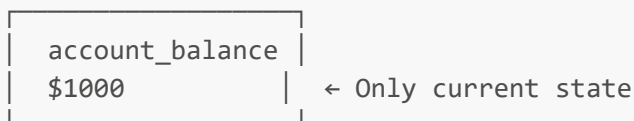
except ClientError as e:
    if e.response['Error']['Code'] == 'TransactionCanceledException':
        # Transaction failed - likely insufficient balance
        raise InsufficientFundsError()
    raise

```

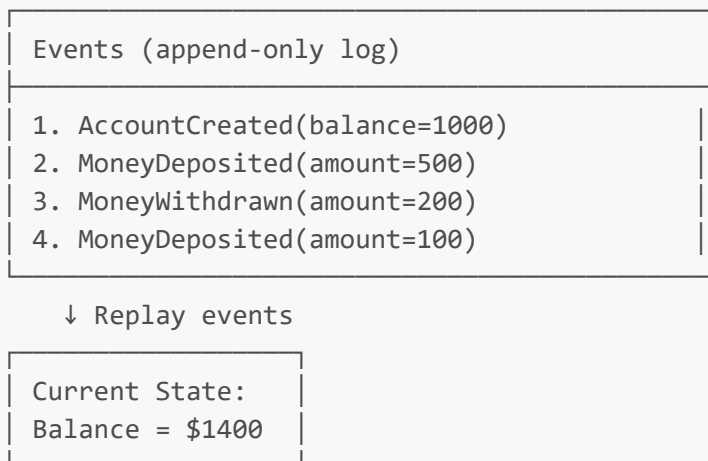
Solution 5: Event Sourcing Pattern

Concept: Store events instead of state, replay events to get current state.

Traditional State Storage:



Event Sourcing:



Why Event Sourcing is Idempotent:

- Events are immutable (append-only)
- Same event ID = same event (idempotent)
- Replay is deterministic

Implementation:

```

from dataclasses import dataclass
from datetime import datetime
from enum import Enum

class EventType(Enum):
    ACCOUNT_CREATED = "account_created"
    MONEY_DEPOSITED = "money_deposited"
    MONEY_WITHDRAWN = "money_withdrawn"

@dataclass
class Event:
    event_id: str # Idempotency key
    event_type: EventType
    account_id: str
    amount: float
    timestamp: datetime
    metadata: dict

class EventStore:
    def __init__(self, db):
        self.db = db

    def append_event(self, event: Event) -> bool:
        """Append event (idempotent)"""

        try:
            self.db.execute(
                """
                INSERT INTO events (
                    event_id,
                    event_type,
                    account_id,
                    amount,
                    timestamp,
                    metadata
                ) VALUES (%s, %s, %s, %s, %s, %s)
                """,
                (
                    event.event_id,
                    event.event_type.value,
                    event.account_id,
                    event.amount,
                    event.timestamp,
                    json.dumps(event.metadata)
                )
            )
            return True # Event appended

        except UniqueViolationError:
            # Event ID already exists - idempotent ✓
            return False # Event already exists

```

```

def get_events(self, account_id: str) -> list[Event]:
    """Get all events for account"""

    rows = self.db.execute(
        """
        SELECT * FROM events
        WHERE account_id = %s
        ORDER BY timestamp ASC
        """,
        (account_id,)
    ).fetchall()

    return [Event(**row) for row in rows]

class AccountAggregate:
    """Rebuild account state from events"""

    def __init__(self, account_id: str, event_store: EventStore):
        self.account_id = account_id
        self.event_store = event_store
        self.balance = 0
        self.version = 0

        # Replay all events
        events = event_store.get_events(account_id)
        for event in events:
            self.apply_event(event)

    def apply_event(self, event: Event):
        """Apply event to current state"""

        if event.event_type == EventType.ACCOUNT_CREATED:
            self.balance = event.amount
        elif event.event_type == EventType.MONEY_DEPOSITED:
            self.balance += event.amount
        elif event.event_type == EventType.MONEY_WITHDRAWN:
            self.balance -= event.amount

        self.version += 1

    def deposit(self, amount: float, idempotency_key: str):
        """Deposit money (idempotent)"""

        event = Event(
            event_id=idempotency_key,
            event_type=EventType.MONEY_DEPOSITED,
            account_id=self.account_id,
            amount=amount,
            timestamp=datetime.utcnow(),
            metadata={}
        )

        appended = self.event_store.append_event(event)

```

```

        if appended:
            self.apply_event(event)
            return {'status': 'deposited', 'new_balance': self.balance}
        else:
            return {'status': 'duplicate', 'balance': self.balance}

# Usage - Idempotent!
event_store = EventStore(db)
account = AccountAggregate('account-123', event_store)

# First call
result = account.deposit(100, idempotency_key='deposit-abc-123')
# {'status': 'deposited', 'new_balance': 1100}

# Retry with same key (network timeout)
result = account.deposit(100, idempotency_key='deposit-abc-123')
# {'status': 'duplicate', 'balance': 1100} # Not deposited twice ✓

```

Event Sourcing Schema:

```

CREATE TABLE events (
    event_id VARCHAR(255) PRIMARY KEY, -- Idempotency key
    event_type VARCHAR(50) NOT NULL,
    aggregate_id VARCHAR(255) NOT NULL,
    aggregate_type VARCHAR(50) NOT NULL,
    event_data JSONB NOT NULL,
    timestamp TIMESTAMP NOT NULL,
    version INT NOT NULL,

    UNIQUE(aggregate_id, version) -- Prevent gaps
);

CREATE INDEX idx_aggregate ON events(aggregate_id, timestamp);

-- Snapshots for performance (optional)
CREATE TABLE snapshots (
    aggregate_id VARCHAR(255) PRIMARY KEY,
    aggregate_type VARCHAR(50) NOT NULL,
    state JSONB NOT NULL,
    version INT NOT NULL,
    timestamp TIMESTAMP NOT NULL
);

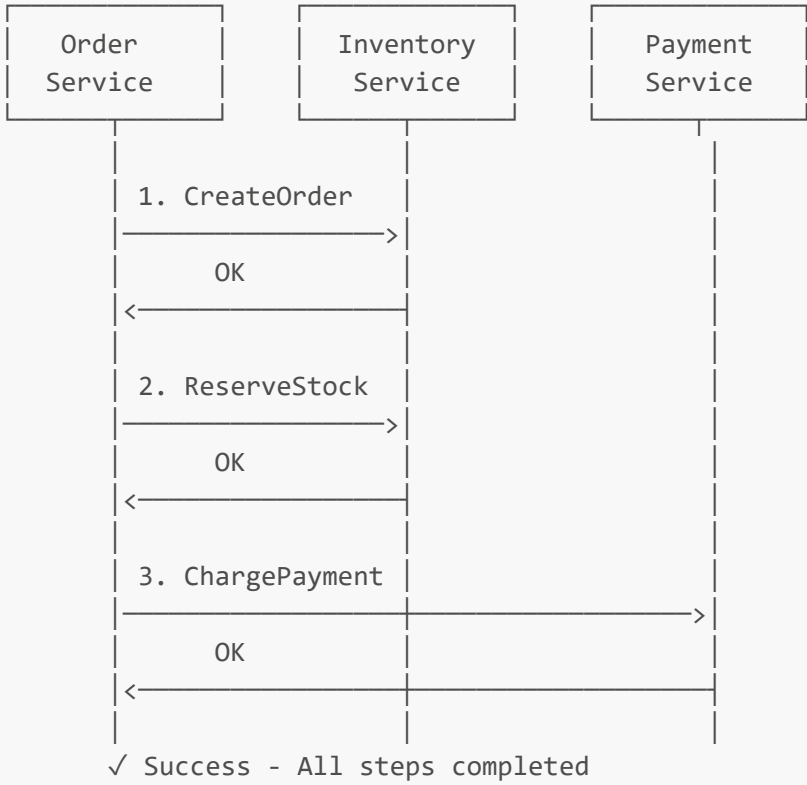
```

Solution 6: Saga Pattern

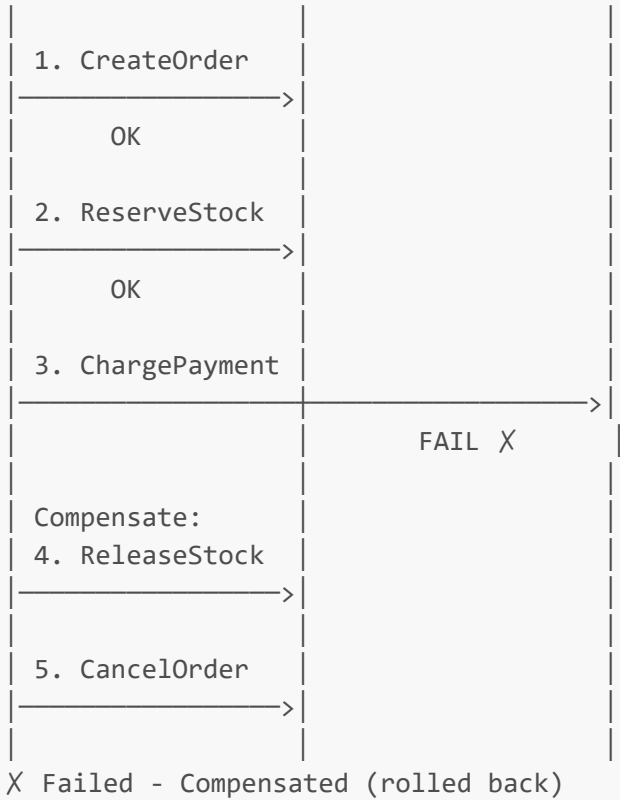
Concept: Distributed transactions across microservices using compensating actions.

Saga: Create Order → Reserve Inventory → Process Payment

Success Flow:



Failure Flow with Compensation:



Implementation:

```
from enum import Enum

class SagaStatus(Enum):
    PENDING = "pending"
```

```

RUNNING = "running"
COMPLETED = "completed"
COMPENSATING = "compensating"
FAILED = "failed"

```

```

class SagaStep:
    def __init__(self, name: str, action: callable, compensation: callable):
        self.name = name
        self.action = action
        self.compensation = compensation

```

```

class Saga:
    """Saga orchestrator with compensation"""

    def __init__(self, saga_id: str, steps: list[SagaStep]):
        self.saga_id = saga_id
        self.steps = steps
        self.completed_steps = []
        self.status = SagaStatus.PENDING

```

```

    async def execute(self):
        """Execute saga steps with compensation on failure"""

        self.status = SagaStatus.RUNNING

        try:
            # Execute each step
            for step in self.steps:
                logger.info(f"Executing step: {step.name}")

                result = await step.action()

                if not result.success:
                    # Step failed - compensate
                    logger.error(f"Step {step.name} failed: {result.error}")
                    await self.compensate()
                    self.status = SagaStatus.FAILED
                    return False

                self.completed_steps.append(step)

            self.status = SagaStatus.COMPLETED
            return True

        except Exception as e:
            logger.error(f"Saga failed: {e}")
            await self.compensate()
            self.status = SagaStatus.FAILED
            return False

```

```

    async def compensate(self):
        """Execute compensation actions in reverse order"""

        self.status = SagaStatus.COMPENSATING

```

```

        # Reverse order compensation
        for step in reversed(self.completed_steps):
            try:
                logger.info(f"Compensating step: {step.name}")
                await step.compensation()
            except Exception as e:
                logger.error(f"Compensation failed for {step.name}: {e}")
                # Continue compensating other steps

# Usage - Order Creation Saga
async def create_order_saga(order_data: dict, idempotency_key: str):
    """Idempotent saga execution"""

    # Check if saga already executed
    existing_saga = await db.fetchone(
        "SELECT * FROM sagas WHERE idempotency_key = %s",
        (idempotency_key,)
    )

    if existing_saga:
        return {'status': 'duplicate', 'saga_id': existing_saga['saga_id']}

    # Define saga steps
    saga = Saga(
        saga_id=f"saga-{uuid.uuid4()}",
        steps=[
            SagaStep(
                name="create_order",
                action=lambda: order_service.create_order(order_data),
                compensation=lambda:
order_service.cancel_order(order_data['order_id'])
            ),
            SagaStep(
                name="reserve_inventory",
                action=lambda:
inventory_service.reserve_stock(order_data['items']),
                compensation=lambda:
inventory_service.release_stock(order_data['items'])
            ),
            SagaStep(
                name="process_payment",
                action=lambda: payment_service.charge(order_data['payment']),
                compensation=lambda: payment_service.refund(order_data['payment'])
            ),
            SagaStep(
                name="send_confirmation",
                action=lambda:
email_service.send_confirmation(order_data['email']),
                compensation=lambda: asyncio.sleep(0) # No compensation needed
            )
        ]
    )

```

```

# Store saga
await db.execute(
    """
    INSERT INTO sagas (saga_id, idempotency_key, status, created_at)
    VALUES (%s, %s, %s, %s)
    """
    (saga.saga_id, idempotency_key, saga.status.value, datetime.utcnow())
)

# Execute saga
success = await saga.execute()

# Update saga status
await db.execute(
    "UPDATE sagas SET status = %s WHERE saga_id = %s",
    (saga.status.value, saga.saga_id)
)

return {'status': saga.status.value, 'saga_id': saga.saga_id}

```

Saga Schema:

```

CREATE TABLE sagas (
    saga_id VARCHAR(255) PRIMARY KEY,
    idempotency_key VARCHAR(255) UNIQUE NOT NULL,
    status VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE saga_steps (
    id SERIAL PRIMARY KEY,
    saga_id VARCHAR(255) REFERENCES sagas(saga_id),
    step_name VARCHAR(100) NOT NULL,
    status VARCHAR(50) NOT NULL,
    result JSONB,
    created_at TIMESTAMP NOT NULL,
    completed_at TIMESTAMP
);

```

Decision Guide: Choosing the Right Strategy

DECISION TREE

```

START: Do you need distributed coordination?
|
|— NO —> Single database?

```

- YES → Use Optimistic Locking (version numbers)
 - ✓ Simple, fast, scales well
 - Use when: Low contention
- NO → Use Pessimistic Locking (SELECT FOR UPDATE)
 - ✓ Guaranteed consistency
 - Use when: High contention, complex transactions
- YES → Multiple services/databases?
 - Low criticality (caching, rate limiting)
 - Redis Simple Lock
 - ✓ Fast, simple
 - X Single point of failure
 - Medium criticality (multi-DC coordination)
 - Redlock (5 Redis instances)
 - ✓ Better fault tolerance
 - X Controversial, clock drift issues
 - High criticality (financial, inventory)
 - ZooKeeper or etcd
 - ✓ Strong consistency, proven
 - X More complex setup
 - Distributed transactions?
 - Saga Pattern
 - ✓ Handles cross-service transactions
 - ✓ Compensating actions
 - X Eventually consistent

COMPARISON TABLE			
Scenario	Recommended	Alternative	Avoid
Single DB, Low contention	Optimistic Locking	DB unique constraints	Distributed locks (overkill)
Single DB, High contention	Pessimistic Locking	State Machine	No locking
Payment processing	Idempotency Keys + DB	Event Sourcing	No idempotency
Distributed, High criticality	ZooKeeper/ etcd	Redlock	Redis single
Distributed, Low criticality	Redis Simple Lock	Application -level	ZooKeeper (too complex)
Cross-service transactions	Saga Pattern	2PC (two-phase commit)	Distributed transactions

Audit trail needed	Event Sourcing	Saga with logging	Mutable state
--------------------	----------------	-------------------	---------------

7. Storage Mechanisms

PostgreSQL Implementation

Schema Design:

```
-- Idempotency keys table
CREATE TABLE idempotency_keys (
    key VARCHAR(255) PRIMARY KEY,
    request_hash VARCHAR(64) NOT NULL, -- SHA256 of request body
    response_code INT,
    response_body JSONB,
    status VARCHAR(50) DEFAULT 'processing',
    created_at TIMESTAMP DEFAULT NOW(),
    expires_at TIMESTAMP NOT NULL,
    locked_until TIMESTAMP
);

CREATE INDEX idx_expires_at ON idempotency_keys(expires_at);
CREATE INDEX idx_locked_until ON idempotency_keys(locked_until);

-- Auto-cleanup expired keys
CREATE OR REPLACE FUNCTION cleanup_expired_keys()
RETURNS void AS $$
BEGIN
    DELETE FROM idempotency_keys
    WHERE expires_at < NOW();
END;
$$ LANGUAGE plpgsql;

-- Schedule cleanup (every hour)
SELECT cron.schedule('cleanup-idempotency', '0 * * * *', 'SELECT
cleanup_expired_keys()');
```

Implementation:

```
class PostgresIdempotency:
    def __init__(self, db, ttl_seconds: int = 86400):
        self.db = db
        self.ttl_seconds = ttl_seconds

    def check_or_create(self, key: str, request_hash: str) -> tuple[bool, dict]:
        """
```

```

    Check if key exists, create if not.
    Returns: (is_duplicate, stored_response)
    """

    # Try to insert key
    result = self.db.execute(
        """
        INSERT INTO idempotency_keys (key, request_hash, expires_at, status)
        VALUES (%s, %s, NOW() + INTERVAL '%s seconds', 'processing')
        ON CONFLICT (key) DO NOTHING
        RETURNING key
        """,
        (key, request_hash, self.ttl_seconds)
    )

    if result.rowcount > 0:
        # New key created - not a duplicate
        return False, None

    # Key exists - check response
    existing = self.db.execute(
        """
        SELECT response_code, response_body, status, request_hash
        FROM idempotency_keys
        WHERE key = %s
        """,
        (key,)
    ).fetchone()

    if existing['request_hash'] != request_hash:
        raise IdempotencyMismatchError(
            "Same key used for different request"
        )

    if existing['status'] == 'completed':
        # Return cached response
        return True, {
            'code': existing['response_code'],
            'body': existing['response_body']
        }
    elif existing['status'] == 'processing':
        # Still processing - client should wait and retry
        raise ProcessingError("Request is being processed")

    return False, None

def store_response(self, key: str, code: int, body: dict):
    """Store successful response"""

    self.db.execute(
        """
        UPDATE idempotency_keys
        SET response_code = %s,
            response_body = %s,

```

```

        status = 'completed'
    WHERE key = %s
    """,
    (code, json.dumps(body), key)
)

```

Redis Implementation

Data Structures:

```

class RedisIdempotency:
    """Redis-based idempotency with multiple data structures"""

    def __init__(self, redis_client, ttl: int = 86400):
        self.redis = redis_client
        self.ttl = ttl

    # Strategy 1: Simple String
    async def check_simple(self, key: str) -> bool:
        """Simple existence check"""
        return await self.redis.exists(f"idem:{key}")

    async def store_simple(self, key: str, value: dict):
        """Store with simple string"""
        await self.redis.setex(
            f"idem:{key}",
            self.ttl,
            json.dumps(value)
        )

    # Strategy 2: Hash (better for partial updates)
    async def check_hash(self, key: str) -> dict:
        """Check using hash"""
        return await self.redis.hgetall(f"idem:{key}")

    async def store_hash(self, key: str, fields: dict):
        """Store using hash"""
        await self.redis.hset(f"idem:{key}", mapping=fields)
        await self.redis.expire(f"idem:{key}", self.ttl)

    # Strategy 3: Sorted Set (for time-based queries)
    async def store_sorted_set(self, key: str, score: float, value: dict):
        """Store in sorted set (score = timestamp)"""
        await self.redis.zadd(
            "idem:keys",
            {key: score}
        )
        await self.redis.setex(f"idem:data:{key}", self.ttl, json.dumps(value))

    async def cleanup_old_keys(self, before_timestamp: float):
        """Remove old keys from sorted set"""

```

```

        removed = await self.redis.zremrangebyscore(
            "idem:keys",
            "-inf",
            before_timestamp
        )
        return removed

```

Redis Lua Scripts (Atomic Operations):

```

-- Atomic check-and-set
local check_and_set = [[
    local key = KEYS[1]
    local value = ARGV[1]
    local ttl = ARGV[2]

    if redis.call("exists", key) == 1 then
        return redis.call("get", key)
    else
        redis.call("setex", key, ttl, value)
        return nil
    end
]]

-- Usage in Python
result = redis.eval(
    check_and_set,
    1, # Number of keys
    f"idem:{idempotency_key}",
    json.dumps(response),
    86400
)

```

DynamoDB Implementation

Table Design:

```

# DynamoDB table structure
table_schema = {
    'TableName': 'idempotency_keys',
    'KeySchema': [
        {'AttributeName': 'idempotency_key', 'KeyType': 'HASH'}
    ],
    'AttributeDefinitions': [
        {'AttributeName': 'idempotency_key', 'AttributeType': 'S'},
        {'AttributeName': 'expires_at', 'AttributeType': 'N'}
    ],
    'GlobalSecondaryIndexes': [
        {
            'IndexName': 'expires_at_index',

```

```

        'KeySchema': [
            {'AttributeName': 'expires_at', 'KeyType': 'HASH'}
        ],
        'Projection': {'ProjectionType': 'ALL'}
    }
],
'BillingMode': 'PAY_PER_REQUEST'
}

# Enable TTL for auto-cleanup
dynamodb.update_time_to_live(
    TableName='idempotency_keys',
    TimeToLiveSpecification={
        'Enabled': True,
        'AttributeName': 'expires_at'
    }
)

```

Implementation:

```

import boto3
from botocore.exceptions import ClientError

class DynamoDBIdempotency:
    def __init__(self, table_name: str, ttl_seconds: int = 86400):
        self.dynamodb = boto3.resource('dynamodb')
        self.table = self.dynamodb.Table(table_name)
        self.ttl_seconds = ttl_seconds

    def check_or_create(self, key: str, request_data: dict) -> tuple[bool, dict]:
        """Check for duplicate or create new entry"""

        try:
            # Try to insert
            self.table.put_item(
                Item={
                    'idempotency_key': key,
                    'request_data': request_data,
                    'status': 'processing',
                    'created_at': int(time.time()),
                    'expires_at': int(time.time()) + self.ttl_seconds
                },
                ConditionExpression='attribute_not_exists(idempotency_key)'
            )

            return False, None # New request

        except ClientError as e:
            if e.response['Error']['Code'] == 'ConditionalCheckFailedException':
                # Key exists - get existing response
                response = self.table.get_item(
                    Key={'idempotency_key': key}

```

```

        )

        item = response.get('Item', {})

        if item.get('status') == 'completed':
            return True, item.get('response_data')
        else:
            raise ProcessingError("Request is being processed")
        raise

    def store_response(self, key: str, response_data: dict):
        """Store response"""

        self.table.update_item(
            Key={'idempotency_key': key},
            UpdateExpression='SET #status = :status, response_data = :data',
            ExpressionAttributeNames={'#status': 'status'},
            ExpressionAttributeValues={
                ':status': 'completed',
                ':data': response_data
            }
        )

```

MongoDB Implementation

Schema:

```

// MongoDB schema
db.createCollection("idempotency_keys", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["key", "request_hash", "expires_at"],
            properties: {
                key: { bsonType: "string" },
                request_hash: { bsonType: "string" },
                response_code: { bsonType: "int" },
                response_body: { bsonType: "object" },
                status: { enum: ["processing", "completed", "failed"] },
                created_at: { bsonType: "date" },
                expires_at: { bsonType: "date" }
            }
        }
    }
});

// Indexes
db.idempotency_keys.createIndex({ key: 1 }, { unique: true });
db.idempotency_keys.createIndex({ expires_at: 1 }, { expireAfterSeconds: 0 });

```

Implementation:

```

from pymongo import MongoClient
from pymongo.errors import DuplicateKeyError

class MongoDBIdempotency:
    def __init__(self, mongo_uri: str, ttl_seconds: int = 86400):
        self.client = MongoClient(mongo_uri)
        self.db = self.client.idempotency_db
        self.collection = self.db.idempotency_keys
        self.ttl_seconds = ttl_seconds

    def check_or_create(self, key: str, request_hash: str) -> tuple[bool, dict]:
        """Check for duplicate or create new entry"""

        try:
            # Try to insert
            self.collection.insert_one({
                'key': key,
                'request_hash': request_hash,
                'status': 'processing',
                'created_at': datetime.utcnow(),
                'expires_at': datetime.utcnow() +
timedelta(seconds=self.ttl_seconds)
            })

            return False, None # New request

        except DuplicateKeyError:
            # Key exists - get existing
            existing = self.collection.find_one({'key': key})

            if existing['request_hash'] != request_hash:
                raise IdempotencyMismatchError("Different request with same key")

            if existing['status'] == 'completed':
                return True, {
                    'code': existing.get('response_code'),
                    'body': existing.get('response_body')
                }
            else:
                raise ProcessingError("Request is being processed")

    def store_response(self, key: str, code: int, body: dict):
        """Store response"""

        self.collection.update_one(
            {'key': key},
            {
                '$set': {
                    'response_code': code,
                    'response_body': body,
                    'status': 'completed'
                }
            }
        )

```

```
        }  
    }  
)
```

8. Edge Cases & Challenges

Challenge 1: Race Conditions

Problem:

Two requests with same idempotency key arrive simultaneously

Time	Request A	Request B
T1	Check key: Not found	Check key: Not found
T2	Process payment	Process payment
T3	Store result	Store result
	✗ BOTH PROCESSED!	

Solution: Database Constraints + Locks

```
def handle_concurrent_requests(idempotency_key: str):  
    """Handle concurrent requests safely"""  
  
    # Use database lock or unique constraint  
    try:  
        # PostgreSQL advisory lock  
        db.execute("SELECT pg_advisory_xact_lock(hashtext(%s))",  
            (idempotency_key,))  
  
        # Check if exists (within lock)  
        existing = db.query(  
            "SELECT * FROM idempotency_keys WHERE key = %s",  
            (idempotency_key,) )  
  
        if existing:  
            return existing  
  
        # Process (only one request reaches here)  
        result = process_payment()  
  
        # Store result  
        db.execute(  
            "INSERT INTO idempotency_keys (key, response) VALUES (%s, %s)",  
            (idempotency_key, result) )
```

```

        return result

    finally:
        # Lock released automatically at transaction end
        pass

```

Challenge 2: Clock Skew in Distributed Systems

Problem:

```

Server A: time = 12:00:00
Server B: time = 12:00:30 (30 seconds ahead)

TTL = 60 seconds

Request to A at 11:59:45 (A's clock)
→ Creates key with expiry = 12:59:45

Request to B at 12:00:10 (B's clock)
→ Key appears expired (12:00:10 > 11:59:45 in B's view)
→ Duplicate processing ✗

```

Solution: Logical Clocks + Longer TTLs

```

# Use version numbers instead of timestamps
class LogicalClockIdempotency:
    def __init__(self):
        self.clock = 0

    def create_key(self, idempotency_key: str):
        """Create key with logical clock"""

        self.clock += 1

        db.execute(
            """
            INSERT INTO idempotency_keys (key, logical_clock, physical_time)
            VALUES (%s, %s, %s)
            """,
            (idempotency_key, self.clock, datetime.utcnow())
        )

    def is_duplicate(self, idempotency_key: str, max_clock_drift: int = 1000):
        """Check with clock drift tolerance"""

        existing = db.query(
            "SELECT logical_clock FROM idempotency_keys WHERE key = %s",
            (idempotency_key,)
        )

```

```

        if existing and (self.clock - existing['logical_clock']) <
max_clock_drift:
            return True

        return False

```

Challenge 3: TTL and Expiration

Problem:

Client stores idempotency key: "order-123"
 Server TTL: 24 hours

Scenario:

Day 1, 10:00 AM: Client sends request (key stored)

Day 1, 10:01 AM: Network timeout, client retries → OK (key exists)

Day 2, 10:01 AM: Client retries again → Key expired → DUPLICATE! ✗

Solution: Configurable TTL + Warning

```

class SmartTTLIdempotency:
    def __init__(self, min_ttl: int = 86400, max_ttl: int = 604800):
        self.min_ttl = min_ttl # 24 hours
        self.max_ttl = max_ttl # 7 days

    def calculate_ttl(self, request_type: str) -> int:
        """Calculate TTL based on request type"""

        ttl_map = {
            'payment': 86400, # 24 hours (payments settle quickly)
            'order': 259200, # 3 days (orders can be modified)
            'webhook': 604800 # 7 days (webhooks retry for days)
        }

        return ttl_map.get(request_type, self.min_ttl)

    def check_near_expiry(self, key: str) -> bool:
        """Warn if key is near expiry"""

        result = db.query(
            "SELECT expires_at FROM idempotency_keys WHERE key = %s",
            (key,)
        )

        if result:
            time_remaining = result['expires_at'] - datetime.utcnow()

            if time_remaining < timedelta(hours=1):

```

```

        logger.warning(
            f"Idempotency key {key} expires in {time_remaining}"
        )
        return True

    return False

```

Challenge 4: Storage Cleanup

Problem: Old idempotency keys accumulate, wasting storage.

Solution: Automated Cleanup

```

# Strategy 1: Background job
class IdempotencyCleanup:
    def __init__(self, db, batch_size: int = 1000):
        self.db = db
        self.batch_size = batch_size

    def cleanup_expired(self):
        """Delete expired keys in batches"""

        while True:
            deleted = self.db.execute(
                """
                DELETE FROM idempotency_keys
                WHERE id IN (
                    SELECT id FROM idempotency_keys
                    WHERE expires_at < NOW()
                    LIMIT %s
                )
                """,
                (self.batch_size,)
            ).rowcount

            if deleted == 0:
                break

            logger.info(f"Deleted {deleted} expired keys")
            time.sleep(0.1) # Avoid blocking database

# Strategy 2: Database-level TTL (PostgreSQL)
CREATE EXTENSION IF NOT EXISTS pg_cron;

SELECT cron.schedule(
    'cleanup-idempotency-keys',
    '* /5 * * * *', -- Every 5 minutes
    $$DELETE FROM idempotency_keys WHERE expires_at < NOW()$$
);

# Strategy 3: Redis automatic expiration

```

```
# Keys automatically deleted when TTL expires (no cleanup needed)

# Strategy 4: DynamoDB TTL
# Enable TTL on expires_at attribute (automatic cleanup)
```

Challenge 5: Request Body Mismatch

Problem:

```
Request 1: POST /payment { "amount": 100, "key": "abc123" }
Request 2: POST /payment { "amount": 200, "key": "abc123" } # Different amount!

Should this be idempotent?
```

Solution: Hash Request Body

```
import hashlib
import json

def hash_request(data: dict) -> str:
    """Create deterministic hash of request"""

    # Sort keys for consistency
    sorted_json = json.dumps(data, sort_keys=True)
    return hashlib.sha256(sorted_json.encode()).hexdigest()

def check_idempotency_with_hash(key: str, request_data: dict):
    """Verify request body matches"""

    request_hash = hash_request(request_data)

    existing = db.query(
        "SELECT request_hash FROM idempotency_keys WHERE key = %s",
        (key,)
    )

    if existing:
        if existing['request_hash'] != request_hash:
            raise IdempotencyMismatchError(
                f"Key {key} used for different request. "
                f"Expected hash: {existing['request_hash']}, "
                f"Got: {request_hash}"
            )

        # Hash matches - return cached response
        return get_cached_response(key)

    # New request - store with hash
    store_with_hash(key, request_hash, request_data)
```

Challenge 6: Partial Failures

Problem:

Payment processing steps:

1. Validate card ✓
2. Reserve amount ✓
3. Charge card X (fails)

If client retries with same idempotency key:

- Should we retry from step 3?
- Or return error immediately?

Solution: State Machine with Checkpoints

```
class CheckpointedIdempotency:
    """Idempotency with process checkpoints"""

    def __init__(self, db):
        self.db = db

    def process_with_checkpoints(self, key: str, steps: list):
        """Process with ability to resume from checkpoint"""

        # Get existing state
        state = self.db.query(
            "SELECT checkpoint, status FROM idempotency_keys WHERE key = %s",
            (key,)
        )

        start_from = 0
        if state:
            if state['status'] == 'completed':
                # Already completed
                return get_result(key)
            elif state['status'] == 'failed':
                # Previous failure - can retry
                start_from = state['checkpoint']

        # Execute steps starting from checkpoint
        for i in range(start_from, len(steps)):
            step = steps[i]

            try:
                result = step.execute()

                # Update checkpoint
                self.db.execute(
                    """
```

```

        INSERT INTO idempotency_keys (key, checkpoint, status)
        VALUES (%s, %s, 'processing')
        ON CONFLICT (key) DO UPDATE
        SET checkpoint = %s
        """
        (key, i, i)
    )

    except Exception as e:
        # Mark as failed at this checkpoint
        self.db.execute(
            """
            UPDATE idempotency_keys
            SET status = 'failed', checkpoint = %s, error = %s
            WHERE key = %s
            """
            (i, str(e), key)
        )
        raise

    # All steps completed
    self.db.execute(
        "UPDATE idempotency_keys SET status = 'completed' WHERE key = %s",
        (key,)
    )

```

Last updated: August 18, 2026 'completed' WHERE key = %s """, (code, json.dumps(body), key))

Usage

```
pg_idem = PostgresIdempotency(db, ttl_seconds=86400)
```

```
@app.post("/api/payments") async def create_payment(payment: PaymentRequest, idempotency_key: str =
Header(...)): request_hash = hashlib.sha256(json.dumps(payment.dict()).encode()).hexdigest()
```

```

is_duplicate, cached = pg_idem.check_or_create(idempotency_key, request_hash)

if is_duplicate:
    return JSONResponse(content=cached['body'], status_code=cached['code'])

try:
    result = await process_payment(payment)
    pg_idem.store_response(idempotency_key, 200, result)
    return result
except Exception as e:
    # Don't cache errors
    raise

```

```

### Redis Implementation

**Data Structure:**
```python
import redis
import json
from datetime import timedelta

class RedisIdempotency:
 """Redis-based idempotency with fast lookups"""

 def __init__(self, redis_client, ttl: timedelta = timedelta(hours=24)):
 self.redis = redis_client
 self.ttl = int(ttl.total_seconds())

 def check(self, key: str) -> tuple[bool, dict]:
 """Check if request already processed"""

 # Get cached response
 cached = self.redis.get(f"idem:response:{key}")

 if cached:
 response = json.loads(cached)
 return True, response

 # Check if processing
 processing = self.redis.get(f"idem:processing:{key}")
 if processing:
 raise ProcessingError("Request is being processed")

 return False, None

 def mark_processing(self, key: str) -> bool:
 """Mark request as processing (acquire lock)"""

 acquired = self.redis.set(
 f"idem:processing:{key}",
 "1",
 nx=True, # Only if not exists
 ex=60 # Lock expires after 60 seconds
)

 return bool(acquired)

 def store_response(self, key: str, response: dict):
 """Store response and release processing lock"""

 # Store response
 self.redis.setex(
 f"idem:response:{key}",
 self.ttl,

```

```

 json.dumps(response)
)

 # Release processing lock
 self.redis.delete(f"idem:processing:{key}")

def clear_processing(self, key: str):
 """Clear processing lock (on error)"""
 self.redis.delete(f"idem:processing:{key}")

Usage
redis_idem = RedisIdempotency(redis.Redis(), ttl=timedelta(hours=24))

@app.post("/api/orders")
async def create_order(order: OrderRequest, idempotency_key: str = Header(...)):
 # Check for duplicate
 is_duplicate, cached = redis_idem.check(idempotency_key)
 if is_duplicate:
 return cached

 # Acquire processing lock
 if not redis_idem.mark_processing(idempotency_key):
 raise ConflictError("Request is being processed")

 try:
 # Process order
 result = await process_order(order)

 # Cache response
 redis_idem.store_response(idempotency_key, result)

 return result

 except Exception as e:
 # Clear lock on error (allow retry)
 redis_idem.clear_processing(idempotency_key)
 raise

```

### Redis Lua Script (Atomic Operations):

```

-- check_and_lock.lua
-- Atomically check for cached response and acquire lock

local response_key = "idem:response:" .. KEYS[1]
local lock_key = "idem:processing:" .. KEYS[1]
local ttl = tonumber(ARGV[1])

-- Check for cached response
local response = redis.call("GET", response_key)
if response then
 return {"cached", response}
end

```

```
-- Check if processing
local processing = redis.call("GET", lock_key)
if processing then
 return {"processing", nil}
end

-- Acquire lock
redis.call("SETEX", lock_key, 60, "1")
return {"new", nil}
```

```
Load Lua script
check_and_lock_script = redis_client.register_script(lua_script)

Execute atomically
result = check_and_lock_script(keys=[idempotency_key], args=[ttl])
status, data = result

if status == b"cached":
 return json.loads(data)
elif status == b"processing":
 raise ProcessingError()
else:
 # Process request
 pass
```

## DynamoDB Implementation

```
import boto3
from datetime import datetime, timedelta

class DynamoDBIdempotency:
 """DynamoDB idempotency with conditional writes"""

 def __init__(self, table_name: str, ttl_hours: int = 24):
 self.dynamodb = boto3.resource('dynamodb')
 self.table = self.dynamodb.Table(table_name)
 self.ttl_hours = ttl_hours

 def check_or_create(self, key: str, request_hash: str) -> tuple[bool, dict]:
 """Check if key exists, create if not"""

 ttl = int((datetime.utcnow() +
timedelta(hours=self.ttl_hours)).timestamp())

 try:
 # Try to create item
 self.table.put_item(
 Item={
```

```

 'idempotency_key': key,
 'request_hash': request_hash,
 'status': 'processing',
 'ttl': ttl,
 'created_at': datetime.utcnow().isoformat()
 },
 ConditionExpression='attribute_not_exists(idempotency_key)'
)

Key created - not a duplicate
return False, None

except ClientError as e:
 if e.response['Error']['Code'] == 'ConditionalCheckFailedException':
 # Key exists - get item
 response = self.table.get_item(
 Key={'idempotency_key': key}
)

 item = response['Item']

 # Verify request hash matches
 if item['request_hash'] != request_hash:
 raise IdempotencyMismatchError(
 "Same key used for different request"
)

 if item['status'] == 'completed':
 return True, {
 'code': item['response_code'],
 'body': item['response_body']
 }
 else:
 raise ProcessingError("Request is being processed")
 raise

def store_response(self, key: str, code: int, body: dict):
 """Store successful response"""

 self.table.update_item(
 Key={'idempotency_key': key},
 UpdateExpression='SET #status = :status, response_code = :code,
response_body = :body',
 ExpressionAttributeNames={'#status': 'status'},
 ExpressionAttributeValues={
 ':status': 'completed',
 ':code': code,
 ':body': body
 }
)

DynamoDB Table Schema
"""
aws dynamodb create-table \

```

```

--table-name idempotency_keys \
--attribute-definitions \
 AttributeName=idempotency_key,AttributeType=S \
--key-schema \
 AttributeName=idempotency_key,KeyType=HASH \
--provisioned-throughput \
 ReadCapacityUnits=5,WriteCapacityUnits=5 \
--time-to-live-specification \
 Enabled=true,AttributeName=ttl
"""

```

## MongoDB Implementation

```

from pymongo import MongoClient
from datetime import datetime, timedelta

class MongoDBIdempotency:
 """MongoDB idempotency with unique indexes"""

 def __init__(self, mongo_uri: str, db_name: str, ttl_hours: int = 24):
 self.client = MongoClient(mongo_uri)
 self.db = self.client[db_name]
 self.collection = self.db.idempotency_keys
 self.ttl_hours = ttl_hours

 # Create indexes
 self.collection.create_index("idempotency_key", unique=True)
 self.collection.create_index("expires_at", expireAfterSeconds=0) # TTL
index

 def check_or_create(self, key: str, request_hash: str) -> tuple[bool, dict]:
 """Check if key exists, create if not"""

 expires_at = datetime.utcnow() + timedelta(hours=self.ttl_hours)

 try:
 # Try to insert
 self.collection.insert_one({
 'idempotency_key': key,
 'request_hash': request_hash,
 'status': 'processing',
 'expires_at': expires_at,
 'created_at': datetime.utcnow()
 })

 return False, None

 except DuplicateKeyError:
 # Key exists - get document
 doc = self.collection.find_one({'idempotency_key': key})

```

```

 # Verify request hash
 if doc['request_hash'] != request_hash:
 raise IdempotencyMismatchError(
 "Same key used for different request"
)

 if doc['status'] == 'completed':
 return True, {
 'code': doc['response_code'],
 'body': doc['response_body']
 }
 else:
 raise ProcessingError("Request is being processed")

 def store_response(self, key: str, code: int, body: dict):
 """Store successful response"""

 self.collection.update_one(
 {'idempotency_key': key},
 {
 '$set': {
 'status': 'completed',
 'response_code': code,
 'response_body': body,
 'completed_at': datetime.utcnow()
 }
 }
)

Usage
mongo_idem = MongoDBIdempotency(
 mongo_uri="mongodb://localhost:27017",
 db_name="myapp",
 ttl_hours=24
)

@app.post("/api/users")
async def create_user(user: UserRequest, idempotency_key: str = Header(...)):
 request_hash = hashlib.sha256(json.dumps(user.dict()).encode()).hexdigest()

 is_duplicate, cached = mongo_idem.check_or_create(idempotency_key,
 request_hash)

 if is_duplicate:
 return JSONResponse(content=cached['body'], status_code=cached['code'])

 try:
 result = await create_user_in_db(user)
 mongo_idem.store_response(idempotency_key, 201, result)
 return result
 except Exception as e:
 raise

```

## 8. Edge Cases & Challenges

### Challenge 1: Race Conditions

**Problem:** Two requests with same idempotency key arrive simultaneously.

Timeline:

T1: Request A checks key → Not found

T2: Request B checks key → Not found (A hasn't inserted yet)

T3: Request A inserts key and processes

T4: Request B inserts key and processes → DUPLICATE! X

### Solution: Atomic Check-and-Set

```
PostgreSQL - Atomic insert
result = db.execute(
 """
 INSERT INTO idempotency_keys (key, status)
 VALUES (%s, 'processing')
 ON CONFLICT (key) DO NOTHING
 RETURNING key
 """,
 (idempotency_key,)
)

if result.rowcount == 0:
 # Another request got the lock
 # Wait and check for result
 time.sleep(0.1)
 return get_cached_response(idempotency_key)

Redis - SETNX (atomic)
acquired = redis.set(f"idem:lock:{key}", "1", nx=True, ex=60)
if not acquired:
 # Another request is processing
 raise ProcessingError()
```

### Challenge 2: Clock Skew

**Problem:** Distributed systems have clock differences.

Server A (clock ahead): Creates key with expires\_at = 2026-08-18 13:00:00  
Server B (clock behind): Checks at 2026-08-18 12:59:50  
→ Key appears expired on B, but not on A!

### Solutions:

## 1. Use Logical Clocks (Version Numbers):

```
Instead of timestamps, use version numbers
CREATE TABLE idempotency_keys (
 key VARCHAR(255) PRIMARY KEY,
 version INT NOT NULL DEFAULT 1,
 created_version INT NOT NULL, -- Version when created
 ttl_seconds INT NOT NULL -- Relative TTL
);

Check expiration based on version difference, not time
current_version = get_cluster_version()
if (current_version - key.created_version) > (key.ttl_seconds /
version_increment_interval):
 # Expired
 pass
```

## 2. Use Centralized Time (etcd lease):

```
// etcd handles time centrally
lease, err := client.Grant(ctx, 60) // 60 seconds
_, err = client.Put(ctx, key, value, clientv3.WithLease(lease.ID))

// All nodes see consistent expiration
```

## 3. Use Relative Timeouts:

```
Store creation time + TTL, not absolute expiration
{
 'key': 'abc123',
 'created_at': 1692356400,
 'ttl_seconds': 3600
}

Check expiration
if time.time() - record['created_at'] > record['ttl_seconds']:
 # Expired
 pass
```

## Challenge 3: TTL Management

**Problem:** How long should idempotency keys live?

```
Too Short (1 hour):
→ Legitimate retries after 2 hours → Duplicate processing X
```

Too Long (30 days):

- Storage costs increase
- Stale keys accumulate

Recommended TTL by Use Case:

Use Case	Recommended TTL	Reason
Payment processing	24-48 hours	Most retries within 1 day
Order creation	1-7 days	Customer may retry later
Webhook processing	7-14 days	Provider retry schedules
Email sending	24 hours	Short retry window
API requests (general)	24 hours	Industry standard
Financial transactions	90 days	Regulatory requirements

Adaptive TTL:

```
def calculate_ttl(operation_type: str, amount: float = None) -> int:
 """Calculate TTL based on operation criticality"""

 if operation_type == "payment":
 if amount and amount > 10000:
 return 86400 * 7 # 7 days for high-value payments
 return 86400 * 2 # 2 days for regular payments

 elif operation_type == "order":
 return 86400 * 3 # 3 days

 elif operation_type == "webhook":
 return 86400 * 7 # 7 days

 else:
 return 86400 # Default: 24 hours
```

Challenge 4: Storage Cleanup

**Problem:** Expired keys accumulate, wasting storage.

**Solutions:**

1. Database TTL (Automatic):

```
-- PostgreSQL with pg_cron
CREATE EXTENSION pg_cron;
```

```
SELECT cron.schedule(
 'cleanup-idempotency-keys',
 '0 * * * *', -- Every hour
 $$DELETE FROM idempotency_keys WHERE expires_at < NOW()$$
);
```

## 2. Redis Expiration (Automatic):

```
Redis handles expiration automatically
redis.setex(key, ttl_seconds, value)
```

## 3. DynamoDB TTL (Automatic):

```
DynamoDB auto-deletes based on ttl attribute
item = {
 'id': 'key123',
 'data': 'value',
 'ttl': int((datetime.utcnow() + timedelta(days=1)).timestamp())
}
```

## 4. Manual Cleanup (Batch Job):

```
async def cleanup_expired_keys():
 """Background job to clean up expired keys"""

 while True:
 try:
 # Delete expired keys in batches
 deleted = await db.execute(
 """
 DELETE FROM idempotency_keys
 WHERE expires_at < NOW()
 AND id IN (
 SELECT id FROM idempotency_keys
 WHERE expires_at < NOW()
 LIMIT 1000
)
 """
)

 logger.info(f"Deleted {deleted.rowcount} expired keys")

 # Wait before next batch
 await asyncio.sleep(60) # 1 minute

 except Exception as e:
 logger.error(f"Cleanup failed: {e}")
```

```

 await asyncio.sleep(300) # 5 minutes on error

Start background task
asyncio.create_task(cleanup_expired_keys())

```

## Challenge 5: Request Body Mismatch

**Problem:** Same idempotency key used for different requests.

```

Request 1: POST /payments
Idempotency-Key: abc123
Body: { "amount": 100, "currency": "USD" }

Request 2: POST /payments
Idempotency-Key: abc123 (same key!)
Body: { "amount": 200, "currency": "EUR" } (different body!)

Should this be accepted? X No!

```

## Solution: Request Fingerprinting

```

import hashlib
import json

def compute_request_fingerprint(request_body: dict, headers: dict = None) -> str:
 """Compute request fingerprint for comparison"""

 # Sort dict keys for consistent hashing
 canonical_body = json.dumps(request_body, sort_keys=True)

 # Include relevant headers if needed
 if headers:
 relevant_headers = {
 k: v for k, v in headers.items()
 if k.lower() in ['content-type', 'user-agent']
 }
 canonical_body += json.dumps(relevant_headers, sort_keys=True)

 # Compute SHA256 hash
 return hashlib.sha256(canonical_body.encode()).hexdigest()

@app.post("/payments")
async def create_payment(
 payment: PaymentRequest,
 idempotency_key: str = Header(...),
 request: Request
):
 # Compute fingerprint
 request_fingerprint = compute_request_fingerprint(payment.dict())

```

```

Check existing key
existing = await db.fetchone(
 "SELECT request_fingerprint FROM idempotency_keys WHERE key = %s",
 (idempotency_key,)
)

if existing:
 if existing['request_fingerprint'] != request_fingerprint:
 # Same key, different request!
 raise HTTPException(
 status_code=422,
 detail="Idempotency key already used for different request"
)

 # Same key, same request - return cached response
 return get_cached_response(idempotency_key)

Store fingerprint with key
await db.execute(
 """
 INSERT INTO idempotency_keys (key, request_fingerprint, ...)
 VALUES (%s, %s, ...)
 """,
 (idempotency_key, request_fingerprint, ...)
)

Process payment
return await process_payment(payment)

```

## Challenge 6: Partial Failures

**Problem:** Operation partially succeeds, then fails.

Example: Create order

1. Create order record ✓
2. Reserve inventory ✓
3. Charge payment X (network timeout)

What state to store for idempotency?

## Solution: Transaction Boundaries

```

@app.post("/orders")
async def create_order(order: OrderRequest, idempotency_key: str = Header(...)):
 # Check idempotency first
 is_duplicate, cached = check_idempotency(idempotency_key)
 if is_duplicate:
 return cached

```

```

try:
 # Use database transaction for atomic operations
 async with db.transaction():
 # All or nothing
 order_id = await db.create_order(order)
 await db.reserve_inventory(order.items)
 payment_result = await payment_gateway.charge(order.payment)

 # Only store idempotency key after FULL success
 await store_idempotency_result(idempotency_key, {
 'order_id': order_id,
 'status': 'completed'
 })

 return {'order_id': order_id, 'status': 'completed'}

except PaymentError as e:
 # Transaction rolled back automatically
 # Don't store idempotency key - allow retry
 raise HTTPException(502, f"Payment failed: {e}")

except Exception as e:
 # Don't store idempotency key on errors
 raise

```

## 9. Testing Idempotency

### Unit Tests

```

import pytest
from unittest.mock import Mock, patch

class TestIdempotency:
 """Test idempotent payment processing"""

 @pytest.fixture
 def idempotency_service(self):
 return IdempotencyService(db=Mock(), ttl_hours=24)

 def test_first_request_processes_payment(self, idempotency_service):
 """First request should process payment"""

 # Arrange
 key = "test-key-123"
 payment = {"amount": 100, "currency": "USD"}

 # Act
 result = idempotency_service.process_payment(key, payment)

```

```

 # Assert
 assert result['status'] == 'success'
 assert result['amount'] == 100

def test_duplicate_request_returns_cached_response(self, idempotency_service):
 """Duplicate request should return cached response"""

 # Arrange
 key = "test-key-123"
 payment = {"amount": 100, "currency": "USD"}

 # Act - First request
 result1 = idempotency_service.process_payment(key, payment)

 # Act - Duplicate request
 result2 = idempotency_service.process_payment(key, payment)

 # Assert
 assert result1 == result2
 assert idempotency_service.payment_processor.charge.call_count == 1 #
Only called once

def test_different_body_same_key_raises_error(self, idempotency_service):
 """Same key with different body should raise error"""

 # Arrange
 key = "test-key-123"
 payment1 = {"amount": 100, "currency": "USD"}
 payment2 = {"amount": 200, "currency": "EUR"} # Different!

 # Act & Assert
 idempotency_service.process_payment(key, payment1)

 with pytest.raises(IdempotencyMismatchError):
 idempotency_service.process_payment(key, payment2)

def test_expired_key_allows_reprocessing(self, idempotency_service):
 """Expired key should allow reprocessing"""

 # Arrange
 key = "test-key-123"
 payment = {"amount": 100, "currency": "USD"}

 # Act - First request
 result1 = idempotency_service.process_payment(key, payment)

 # Simulate time passing (key expires)
 with patch('time.time', return_value=time.time() + 86400 * 2): # 2 days
later
 # Act - Request with expired key
 result2 = idempotency_service.process_payment(key, payment)

 # Assert

```

```
assert idempotency_service.payment_processor.charge.call_count == 2 #
Called twice
```

## Integration Tests

```
import asyncio
import pytest
from httpx import AsyncClient

@pytest.mark.asyncio
class TestIdempotencyIntegration:
 """Integration tests for idempotent endpoints"""

 async def test_concurrent_requests_same_key(self):
 """Multiple concurrent requests with same key should process once"""

 async with AsyncClient(app=app, base_url="http://test") as client:
 idempotency_key = "concurrent-test-123"
 payment_data = {"amount": 100, "currency": "USD"}

 # Send 10 concurrent requests with same key
 tasks = [
 client.post(
 "/payments",
 json=payment_data,
 headers={"Idempotency-Key": idempotency_key}
)
 for _ in range(10)
]

 responses = await asyncio.gather(*tasks)

 # All should return 200
 assert all(r.status_code == 200 for r in responses)

 # All should return same response
 first_response = responses[0].json()
 assert all(r.json() == first_response for r in responses)

 # Payment should be processed only once
 payment_count = await db.fetchone(
 "SELECT COUNT(*) FROM payments WHERE idempotency_key = %s",
 (idempotency_key,)
)
 assert payment_count['count'] == 1

 async def test_network_retry_scenario(self):
 """Simulate network timeout and retry"""

 async with AsyncClient(app=app, base_url="http://test") as client:
 idempotency_key = "retry-test-456"
```

```

payment_data = {"amount": 500, "currency": "USD"}

First request (simulate timeout on client side)
try:
 response1 = await client.post(
 "/payments",
 json=payment_data,
 headers={"Idempotency-Key": idempotency_key},
 timeout=0.001 # Very short timeout
)
except TimeoutException:
 pass # Expected

Wait for server to finish processing
await asyncio.sleep(1)

Retry with same key
response2 = await client.post(
 "/payments",
 json=payment_data,
 headers={"Idempotency-Key": idempotency_key}
)

Should return cached response
assert response2.status_code == 200

Payment should exist only once
payment_count = await db.fetchone(
 "SELECT COUNT(*) FROM payments WHERE idempotency_key = %s",
 (idempotency_key,)
)
assert payment_count['count'] == 1

```

## Chaos Testing

```

import random
from locust import HttpUser, task, between

class IdempotencyChaosTest(HttpUser):
 """Chaos testing for idempotency under load"""

 wait_time = between(0.1, 0.5)

 def on_start(self):
 """Setup"""
 self.idempotency_keys = [
 f"chaos-key-{i}" for i in range(100)
]

 @task(10)
 def duplicate_requests(self):

```

```

 """Send duplicate requests intentionally"""

 # Pick a key (high chance of collision)
 key = random.choice(self.idempotency_keys[:10])

 self.client.post(
 "/payments",
 json={"amount": 100, "currency": "USD"},
 headers={"Idempotency-Key": key}
)

 @task(5)
 def rapid_fire_same_key(self):
 """Send multiple requests with same key rapidly"""

 key = f"rapid-fire-{random.randint(1, 5)}"

 for _ in range(5):
 self.client.post(
 "/payments",
 json={"amount": 100, "currency": "USD"},
 headers={"Idempotency-Key": key}
)

 @task(2)
 def network_timeout_simulation(self):
 """Simulate network timeout"""

 key = f"timeout-{random.randint(1, 100)}"

 try:
 self.client.post(
 "/payments",
 json={"amount": 100, "currency": "USD"},
 headers={"Idempotency-Key": key},
 timeout=0.1 # Very short timeout
)
 except:
 pass # Expected

Run: locust -f chaos_test.py --host=http://localhost:8000

```

## 10. Best Practices & Anti-patterns

### Best Practices ✓

#### 1. Always Use Idempotency Keys for Mutating Operations

```

✓ Good - Idempotency key required
@app.post("/payments")

```

```
def create_payment(
 payment: PaymentRequest,
 idempotency_key: str = Header(..., description="Required for idempotency")
):
 pass

X Bad - No idempotency protection
@app.post("/payments")
def create_payment(payment: PaymentRequest):
 pass
```

## 2. Generate Keys on Client Side

```
// ✓ Good - Client generates key
const idempotencyKey = `payment-${userId}-${Date.now()}-${Math.random()}`;

await fetch('/api/payments', {
 method: 'POST',
 headers: {
 'Idempotency-Key': idempotencyKey,
 'Content-Type': 'application/json'
 },
 body: JSON.stringify(paymentData)
});

// X Bad - Server generates key (defeats purpose)
// Server can't detect duplicates if client doesn't send key
```

## 3. Store Request Fingerprint

```
✓ Good - Verify request body hasn't changed
store_idempotency_key(
 key=idempotency_key,
 request_hash=compute_hash(request_body),
 response=response
)

X Bad - No verification
store_idempotency_key(key=idempotency_key, response=response)
```

## 4. Don't Cache Errors (Allow Retry)

```
✓ Good - Only cache successful responses
try:
 result = process_payment(payment)
 cache_idempotency_result(key, result) # Cache success
 return result
```

```

except PaymentError as e:
 # Don't cache error - allow retry
 raise

X Bad - Caching errors
try:
 result = process_payment(payment)
except PaymentError as e:
 cache_idempotency_result(key, {'error': str(e)}) # X Don't do this!
 raise

```

## 5. Use Appropriate TTL

```

✓ Good - TTL based on use case
if operation_type == "financial_transaction":
 ttl = timedelta(days=90) # Longer for audit trail
elif operation_type == "email":
 ttl = timedelta(hours=24) # Shorter for emails
else:
 ttl = timedelta(hours=24) # Default

X Bad - One-size-fits-all TTL
ttl = timedelta(hours=24) # For everything

```

## 6. Document Idempotency Behavior

```

@app.post("/payments")
async def create_payment(
 payment: PaymentRequest,
 idempotency_key: str = Header(..., description="Unique key for idempotency")
):
 """
 Create a payment.

 Idempotency:
 - This endpoint is idempotent when using the Idempotency-Key header
 - Duplicate requests with the same key return the original response
 - Keys expire after 24 hours
 - Same key with different request body returns 422 error

 Example:
 ...
 POST /payments
 Headers:
 Idempotency-Key: payment-user123-1692356400-abc
 Body:
 {"amount": 100, "currency": "USD"}
 ...

```

```
"""
pass
```

## Anti-patterns X

### 1. Relying on HTTP Method Alone

```
X Bad - Assuming PUT is always idempotent
@app.put("/user/{user_id}/credits")
def add_credits(user_id: int, credits: int):
 # This increments! Not idempotent!
 db.execute("UPDATE users SET credits = credits + %s WHERE id = %s", (credits,
user_id))

✓ Good - Explicit idempotency key
@app.put("/user/{user_id}/credits")
def add_credits(user_id: int, credits: int, idempotency_key: str = Header(...)):
 # Check idempotency, then increment
 pass
```

### 2. Using Predictable Keys

```
X Bad - Predictable keys
idempotency_key = f"payment-{user_id}" # Same key for all payments!

✓ Good - Unique keys
idempotency_key = f"payment-{user_id}-{timestamp}-{uuid.uuid4()}"
```

### 3. Not Handling Concurrent Requests

```
X Bad - Race condition
def process_payment(idempotency_key):
 existing = db.get(idempotency_key)
 if existing:
 return existing

 # Two requests can both reach here!
 result = charge_payment()
 db.insert(idempotency_key, result)
 return result

✓ Good - Atomic check-and-set
def process_payment(idempotency_key):
 # Atomic insert
 inserted = db.insert_if_not_exists(idempotency_key, status='processing')
 if not inserted:
 return db.get(idempotency_key)
```

```
result = charge_payment()
db.update(idempotency_key, result)
return result
```

#### 4. Infinite Idempotency Keys

```
X Bad - No expiration
redis.set(f"idem:{key}", value) # Lives forever!

✓ Good - Set TTL
redis.setex(f"idem:{key}", 86400, value) # Expires in 24 hours
```

#### 5. Ignoring Request Body Changes

```
X Bad - Same key, different body accepted
def create_payment(idempotency_key, payment):
 existing = db.get_by_key(idempotency_key)
 if existing:
 return existing # Returns regardless of body!

 return process_payment(payment)

✓ Good - Validate request matches
def create_payment(idempotency_key, payment):
 request_hash = compute_hash(payment)
 existing = db.get_by_key(idempotency_key)

 if existing:
 if existing.request_hash != request_hash:
 raise IdempotencyMismatchError()
 return existing.response

 return process_payment(payment)
```

---

## Interview Questions (50)

### Basic Concepts (Q1-10)

#### Q1: What is idempotency in the context of APIs?

**Answer:** Idempotency is the property of an operation where performing it multiple times produces the same result as performing it once. In APIs, an idempotent endpoint can be called repeatedly with the same parameters without causing unintended side effects.

Example:

```
Idempotent: DELETE /users/123
- First call: Deletes user 123
- Second call: User 123 already deleted (same result)
- Nth call: Same state (idempotent ✓)

Non-idempotent: POST /users
- First call: Creates user with ID 1
- Second call: Creates user with ID 2 (different result!)
- Not idempotent X
```

---

## Q2: Which HTTP methods are naturally idempotent?

### Answer:

- **GET** - Reading data (naturally idempotent)
- **PUT** - Replacing entire resource (idempotent)
- **DELETE** - Deleting resource (idempotent)
- **HEAD** - Like GET but no body (idempotent)
- **OPTIONS** - Get allowed methods (idempotent)

### NOT idempotent:

- **POST** - Creating resources (generates new IDs)
- **PATCH** - Can be idempotent or not (depends on operation)

---

## Q3: Why is idempotency important in distributed systems?

### Answer:

1. **Network reliability:** Clients can safely retry failed requests without causing duplicates
2. **At-least-once delivery:** Message queues often deliver messages multiple times
3. **Failure recovery:** Systems can recover from crashes by replaying operations
4. **User experience:** Prevents double-charges, duplicate orders, etc.
5. **System resilience:** Makes systems more fault-tolerant

Real-world example: User clicks "Pay" button, network times out. Without idempotency, retry causes double charge. With idempotency, retry returns original response.

---

## Q4: What is an idempotency key and how is it used?

**Answer:** An idempotency key is a unique identifier sent by the client to ensure an operation is performed only once, even if the request is sent multiple times.

Flow:

- ```
1. Client generates unique key: "payment-abc-123"
2. Client sends request with key in header:
```

```

    POST /payments
    Idempotency-Key: payment-abc-123
3. Server checks if key exists
   - If not: Process request, store result with key
   - If yes: Return stored result (don't reprocess)
4. Client retries with same key → Gets cached response

```

Best practices:

- Client generates key (UUID or timestamp-based)
- Server stores key + request hash + response
- Keys expire after TTL (e.g., 24 hours)

Q5: How do you implement idempotency for POST requests?

Answer: POST is not naturally idempotent, so we use idempotency keys:

```

@app.post("/orders")
async def create_order(
    order: OrderRequest,
    idempotency_key: str = Header(...)
):
    # 1. Check if key exists
    existing = await db.fetchone(
        "SELECT * FROM orders WHERE idempotency_key = %s",
        (idempotency_key,)
    )

    if existing:
        # Return existing order (idempotent)
        return existing

    # 2. Create new order
    try:
        order_id = await db.execute(
            """
            INSERT INTO orders (idempotency_key, user_id, items, total)
            VALUES (%s, %s, %s, %s)
            RETURNING id
            """,
            (idempotency_key, order.user_id, order.items, order.total)
        )

        return {"order_id": order_id, "status": "created"}

    except UniqueViolationError:
        # Another request created it concurrently
        existing = await db.fetchone(
            "SELECT * FROM orders WHERE idempotency_key = %s",
            (idempotency_key,)

```

```
)  
return existing
```

Database schema:

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  idempotency_key VARCHAR(255) UNIQUE NOT NULL,  
  user_id INT,  
  items JSONB,  
  total DECIMAL,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Q6: What are the differences between idempotency and exactly-once delivery?

Answer:

| Aspect | Idempotency | Exactly-Once Delivery |
|----------------|---|---|
| Definition | Same operation multiple times = same result | Operation executed exactly once, never duplicated |
| Guarantee | Application-level guarantee | Infrastructure-level guarantee |
| Implementation | Check if operation already done | Prevent duplicate delivery |
| Complexity | Simpler (application logic) | Complex (distributed transactions) |
| Performance | Fast (check + skip) | Slower (coordination overhead) |
| Example | Idempotent API with keys | Kafka exactly-once semantics |

Key insight: Idempotency is often easier to achieve and more practical than exactly-once delivery.

Q7: How do you handle race conditions with idempotency keys?

Answer: Use atomic database operations to prevent race conditions:

Problem:

```
Time  Request A          Request B  
T1    Check key (not found)  
T2                    Check key (not found)  
T3    Insert key  
T4                    Insert key ← Race condition!
```

Solution 1: Database Unique Constraint

```
CREATE TABLE idempotency_keys (  
    key VARCHAR(255) PRIMARY KEY -- Atomic uniqueness  
);  
  
-- Insert with ON CONFLICT  
INSERT INTO idempotency_keys (key, status)  
VALUES ('abc123', 'processing')  
ON CONFLICT (key) DO NOTHING  
RETURNING key;  
  
-- If rowcount = 0, another request won the race
```

Solution 2: Redis SETNX (atomic)

```
# SET if Not eXists (atomic)  
acquired = redis.set(key, "1", nx=True, ex=60)  
if not acquired:  
    # Another request is processing  
    raise ProcessingError()
```

Solution 3: Optimistic Locking

```
UPDATE idempotency_keys  
SET status = 'completed'  
WHERE key = 'abc123'  
    AND status = 'processing' -- Atomic check-and-update  
    AND version = 1  
RETURNING *;
```

Q8: What should be the TTL for idempotency keys?

Answer: Depends on the use case:

| Use Case | TTL | Reasoning |
|--------------------|-------------|----------------------------|
| Payment processing | 24-48 hours | Most retries within a day |
| Order creation | 1-7 days | User may retry later |
| Webhook processing | 7-14 days | Provider retry schedules |
| Email sending | 24 hours | Short retry window |
| Financial txns | 90 days | Regulatory requirements |
| API requests | 24 hours | Industry standard (Stripe) |

Considerations:

- Too short: Legitimate retries may cause duplicates
- Too long: Storage costs increase, stale keys accumulate
- High-value operations: Longer TTL
- Fast operations: Shorter TTL

Stripe's approach: 24 hours for most operations.

Q9: How do you verify that a duplicate request has the same body?

Answer: Store a fingerprint (hash) of the request body with the idempotency key:

```
import hashlib
import json

def compute_fingerprint(request_body: dict) -> str:
    """Compute consistent hash of request"""
    # Sort keys for consistent JSON
    canonical = json.dumps(request_body, sort_keys=True)
    return hashlib.sha256(canonical.encode()).hexdigest()

@app.post("/payments")
async def create_payment(
    payment: PaymentRequest,
    idempotency_key: str = Header(...)
):
    # Compute fingerprint
    request_hash = compute_fingerprint(payment.dict())

    # Check existing key
    existing = await db.fetchone(
        """
        SELECT request_hash, response FROM idempotency_keys
        WHERE key = %s
        """,
        (idempotency_key,)
    )

    if existing:
        # Verify fingerprint matches
        if existing['request_hash'] != request_hash:
            raise HTTPException(
                status_code=422,
                detail="Idempotency key reused with different request body"
            )

        # Same request - return cached response
        return existing['response']

    # New request - store fingerprint
    await db.execute(
```

```

        """
        INSERT INTO idempotency_keys (key, request_hash, ...)
        VALUES (%s, %s, ...)
        """
        (idempotency_key, request_hash, ...)
    )

    # Process payment
    return await process_payment(payment)

```

Why fingerprinting matters: Same idempotency key with different body = programmer error or malicious attempt.

Q10: Should you cache error responses for idempotency?

Answer: Generally NO - don't cache error responses. Allow clients to retry failures.

Rationale:

Scenario: Payment gateway temporarily down

✗ If you cache the error:

1. Client sends payment request
2. Gateway returns 503 Service Unavailable
3. You cache: { key: "abc123", error: "503" }
4. Client retries (gateway now healthy)
5. You return cached error → Payment never succeeds!

✓ If you don't cache errors:

1. Client sends payment request
2. Gateway returns 503
3. You DON'T cache the error
4. Client retries (gateway now healthy)
5. Payment succeeds ✓

Implementation:

```

try:
    result = await process_payment(payment)

    # Only cache successful responses
    await cache_idempotency_result(key, result)
    return result

except PaymentGatewayError as e:
    # Don't cache - allow retry
    logger.error(f"Payment failed: {e}")
    raise HTTPException(502, "Payment gateway unavailable")

```

Exceptions (cache errors only if):

- Client error (400 Bad Request) - request will never succeed
- Validation error (422 Unprocessable Entity) - request is invalid
- Never cache: 500s, 502s, 503s, 504s (transient errors)

Last updated: August 18, 2026

Advanced Concepts (Q11-Q20)**Q11: How does idempotency work in distributed microservices?**

Answer: In microservices, idempotency requires coordination across services. Use distributed idempotency stores or pass keys through the call chain.

Pattern 1: Shared Idempotency Store

```
Client → API Gateway → Service A → Service B
      (Redis)         ↓         ↓
                  Check key  Check key
```

All services check same Redis for idempotency

Pattern 2: Key Propagation

```
Client sends key → Service A (generates sub-keys) → Service B
                  key: abc123                      key: abc123-serviceB
```

Each service generates derived keys

Implementation:

```
# Service A
async def create_order(order_data, idempotency_key):
    # Check own idempotency
    if await redis.get(f"order:{idempotency_key}"):
        return cached_response

    # Call Service B with derived key
    inventory_key = f"{idempotency_key}:inventory"
    await service_b.reserve_inventory(order_data.items, inventory_key)

    # Call Service C with derived key
    payment_key = f"{idempotency_key}:payment"
    await service_c.process_payment(order_data.payment, payment_key)
```

```
# Cache result
await redis.setex(f"order:{idempotency_key}", 86400, result)
return result
```

Q12: Explain optimistic locking for idempotency.

Answer: Optimistic locking uses version numbers to detect concurrent modifications.

Flow:

1. Read record with version: { balance: 100, version: 5 }
2. Modify locally: balance = 90
3. Update with version check:
UPDATE accounts
SET balance = 90, version = 6
WHERE id = 123 AND version = 5
4. If rowcount = 0: version changed → retry
If rowcount = 1: success

Implementation:

```
def withdraw_optimistic(account_id, amount, max_retries=3):
    for attempt in range(max_retries):
        # Read current state
        account = db.fetchone(
            "SELECT balance, version FROM accounts WHERE id = %s",
            (account_id,)
        )

        if account.balance < amount:
            raise InsufficientFundsError()

        # Try to update with version check
        result = db.execute(
            """
            UPDATE accounts
            SET balance = balance - %s,
                version = version + 1
            WHERE id = %s AND version = %s
            RETURNING balance
            """,
            (amount, account_id, account.version)
        )

        if result.rowcount > 0:
            # Success
            return result.fetchone()['balance']

        # Version mismatch - retry
```

```
        time.sleep(0.01 * (2 ** attempt)) # Exponential backoff

    raise OptimisticLockError("Max retries exceeded")
```

When to use:

- Low contention scenarios
- Reads are much more frequent than writes
- Want to avoid blocking

Q13: What is the difference between Redlock and simple Redis locks?

Answer:

| Feature | Simple Redis Lock | Redlock |
|-----------------|-------------------------|----------------------------|
| Redis instances | 1 | 5 (or more, odd number) |
| Fault tolerance | None (single point) | High (majority quorum) |
| Safety | Not safe if Redis fails | Safer (requires majority) |
| Performance | Faster | Slower (multiple RTTs) |
| Complexity | Low | Medium |
| Use case | Non-critical operations | Medium-critical operations |

Simple Redis Lock:

```
acquired = redis.set("lock:key", "value", nx=True, ex=10)
if acquired:
    # Do work
    redis.delete("lock:key")
```

Redlock:

```
# Try to acquire lock on all 5 Redis instances
acquired_count = 0
for redis_instance in redis_instances:
    if redis_instance.set("lock:key", "value", nx=True, ex=10):
        acquired_count += 1

# Need majority (3 out of 5)
if acquired_count >= 3:
    # Lock acquired
    pass
else:
    # Release all locks and retry
    pass
```

Controversy: Martin Kleppmann argues Redlock is not safe due to clock drift and process pauses. Use ZooKeeper/etcd for critical operations.

Q14: How do you implement idempotency for message queues?

Answer: Message queues often deliver messages multiple times (at-least-once delivery). Make consumers idempotent.

Pattern 1: Store Message IDs

```
async def handle_message(message):
    message_id = message['id']

    # Check if already processed
    if await redis.get(f"processed:{message_id}"):
        logger.info(f"Message {message_id} already processed")
        return # Skip

    try:
        # Process message
        await process_order(message['data'])

        # Mark as processed (TTL: 7 days)
        await redis.setex(f"processed:{message_id}", 604800, "1")

    except Exception as e:
        logger.error(f"Processing failed: {e}")
        raise # Message will be retried
```

Pattern 2: Deduplication Window

```
class MessageDeduplicator:
    def __init__(self, window_seconds=300): # 5 minutes
        self.window = window_seconds
        self.seen = {} # In-memory cache

    def is_duplicate(self, message_id):
        now = time.time()

        # Clean old entries
        self.seen = {
            k: v for k, v in self.seen.items()
            if now - v < self.window
        }

        # Check if seen
        if message_id in self.seen:
            return True
```

```
        # Mark as seen
        self.seen[message_id] = now
        return False

deduplicator = MessageDeduplicator(window_seconds=300)

async def consume_messages():
    async for message in queue:
        if deduplicator.is_duplicate(message.id):
            continue

        await handle_message(message)
```

Pattern 3: Database Unique Constraint

```
CREATE TABLE processed_messages (
    message_id VARCHAR(255) PRIMARY KEY,
    processed_at TIMESTAMP DEFAULT NOW()
);

-- Try to insert
INSERT INTO processed_messages (message_id)
VALUES ('msg-123')
ON CONFLICT (message_id) DO NOTHING;

-- If rowcount = 0, already processed
```

Q15: What are the trade-offs between database and cache for storing idempotency keys?

Answer:

| Aspect | Database (PostgreSQL) | Cache (Redis) |
|-------------|-----------------------------|----------------------------|
| Durability | Durable (survives restarts) | Volatile (lost on restart) |
| Performance | Slower (disk I/O) | Faster (in-memory) |
| Consistency | Strong (ACID) | Eventual |
| TTL | Manual cleanup | Automatic expiration |
| Capacity | Larger | Limited by RAM |
| Cost | Cheaper for storage | Expensive for large data |
| Use case | Long TTL, audit trail | Short TTL, high throughput |

Hybrid Approach:

```

class HybridIdempotency:
    def __init__(self, redis, db):
        self.redis = redis
        self.db = db

    async def check(self, key):
        # 1. Check Redis (fast path)
        cached = await self.redis.get(f"idem:{key}")
        if cached:
            return json.loads(cached)

        # 2. Check Database (slow path)
        db_result = await self.db.fetchone(
            "SELECT response FROM idempotency_keys WHERE key = %s",
            (key,)
        )

        if db_result:
            # Warm cache
            await self.redis.setex(
                f"idem:{key}",
                3600, # 1 hour in cache
                json.dumps(db_result['response'])
            )
            return db_result['response']

        return None

    async def store(self, key, response):
        # Write to both
        await self.redis.setex(f"idem:{key}", 3600, json.dumps(response))
        await self.db.execute(
            """
            INSERT INTO idempotency_keys (key, response, expires_at)
            VALUES (%s, %s, NOW() + INTERVAL '24 hours')
            """,
            (key, json.dumps(response))
        )

```

Recommendation:

- High throughput: Redis
- Long TTL / audit requirements: Database
- Best of both: Hybrid approach

Q16: How do you handle clock skew in distributed idempotency?

Answer: Clock skew can cause issues with TTL calculations in distributed systems.

Problem:

```
Server A (clock ahead): Creates key expires_at = 2026-08-18 13:00:00
Server B (clock behind): Checks at      actual = 2026-08-18 12:59:50
→ Server B thinks key expired, Server A thinks it's valid!
```

Solution 1: Relative Time (TTL from creation)

```
# Instead of absolute expiration
{
    'key': 'abc123',
    'created_at': 1692356400,
    'ttl_seconds': 86400
}

# Check expiration
if current_time() - record['created_at'] > record['ttl_seconds']:
    # Expired
```

Solution 2: Centralized Time (Distributed Coordination)

```
# Use etcd lease (centralized time)
lease, err = etcd_client.Grant(ctx, 60) # 60 seconds
etcd_client.Put(ctx, key, value, clientv3.WithLease(lease.ID))

# All servers see consistent expiration
```

Solution 3: Version Numbers (No Time)

```
# Use logical clocks instead of wall clocks
{
    'key': 'abc123',
    'version': 12345, # Cluster version when created
    'ttl_versions': 1000 # Expire after 1000 version increments
}

# Check expiration
if cluster_version() - record['version'] > record['ttl_versions']:
    # Expired
```

Solution 4: NTP Synchronization

- Use NTP to keep clocks synchronized (± 1 -10ms)
- Add clock drift tolerance to TTL calculations

```
# Add drift buffer
ttl_with_buffer = ttl_seconds + max_clock_drift_seconds
```

Q17: How does Stripe implement idempotency?

Answer: Stripe's idempotency implementation is industry-standard:

Key Features:

1. **Client-generated keys:** Required header `Idempotency-Key`
2. **24-hour TTL:** Keys expire after 24 hours
3. **Request fingerprinting:** Verifies request body hasn't changed
4. **Concurrent request handling:** Uses locking during processing
5. **Only cache successes:** Errors are not cached

Flow:

1. Client generates key: "550e8400-e29b-41d4-a716-446655440000"
2. Client sends request:
POST /v1/charges
Idempotency-Key: 550e8400-...
3. Stripe checks key:
 - Not found: Process charge, store result
 - Found, processing: Return 409 Conflict (retry later)
 - Found, completed: Return cached response
4. Stripe stores:
 - Idempotency key
 - Request parameters (fingerprint)
 - Response status and body
 - Timestamp (for 24h expiration)

Example:

```
# First request
response = stripe.Charge.create(
    amount=2000,
    currency="usd",
    source="tok_visa",
    idempotency_key="order-123-attempt-1"
)
# Returns: charge_id = "ch_abc123"

# Retry (network timeout)
response = stripe.Charge.create(
    amount=2000,
```

```

    currency="usd",
    source="tok_visa",
    idempotency_key="order-123-attempt-1" # Same key!
)
# Returns: Same charge_id = "ch_abc123" (not charged twice)

```

Stripe's Recommendations:

- Generate keys on client side
- Use UUID v4 or similar
- Include operation context: `{resource}-{user_id}-{timestamp}`
- Don't reuse keys for different operations

Q18: What is the CAP theorem trade-off for idempotency systems?

Answer: CAP theorem: You can have at most 2 of: Consistency, Availability, Partition tolerance.

For idempotency:

CP (Consistency + Partition tolerance):

- Example: ZooKeeper, etcd
- Strong consistency guarantee
- May be unavailable during network partitions
- **Use for:** Critical operations (payments, inventory)

```

# ZooKeeper lock - CP system
lock = zk.Lock("/locks/inventory-123")
with lock:
    # Guaranteed only one process executes
    update_inventory()

```

AP (Availability + Partition tolerance):

- Example: Redis, DynamoDB
- Always available
- May return stale data during partitions
- **Use for:** Non-critical operations (analytics, logging)

```

# Redis lock - AP system
acquired = redis.set("lock:key", "1", nx=True, ex=60)
if acquired:
    # May have race conditions during network partition
    update_cache()

```

Trade-off for Idempotency:

High Consistency (CP):

- ✓ No duplicate operations
- ✓ Strong guarantees
- ✗ Lower availability
- ✗ Higher latency

Use: Payments, financial transactions

High Availability (AP):

- ✓ Always responsive
- ✓ Low latency
- ✗ Possible duplicates during partition
- ✗ Weaker guarantees

Use: Analytics, non-critical ops

Recommendation:

- Match consistency requirements to business criticality
- Use CP for financial operations
- Use AP for non-critical operations
- Consider hybrid: AP with eventual consistency checks

Q19: How do you test idempotency in production?

Answer: Use monitoring, tracing, and chaos testing to verify idempotency in production.

1. Distributed Tracing:

```
from opentelemetry import trace

tracer = trace.get_tracer(__name__)

@app.post("/payments")
async def create_payment(payment: PaymentRequest, idempotency_key: str =
Header(...)):
    with tracer.start_as_current_span("create_payment") as span:
        span.set_attribute("idempotency_key", idempotency_key)
        span.set_attribute("amount", payment.amount)

        is_duplicate = check_idempotency(idempotency_key)
        span.set_attribute("is_duplicate", is_duplicate)

        if is_duplicate:
            span.add_event("Returning cached response")
            return get_cached_response(idempotency_key)
```

```
result = await process_payment(payment)
return result
```

2. Metrics:

```
from prometheus_client import Counter, Histogram

idempotency_hits = Counter(
    'idempotency_cache_hits_total',
    'Number of idempotency cache hits',
    ['endpoint']
)

idempotency_misses = Counter(
    'idempotency_cache_misses_total',
    'Number of idempotency cache misses',
    ['endpoint']
)

@app.post("/payments")
async def create_payment(payment: PaymentRequest, idempotency_key: str =
Header(...)):
    is_duplicate = check_idempotency(idempotency_key)

    if is_duplicate:
        idempotency_hits.labels(endpoint='/payments').inc()
        return get_cached_response(idempotency_key)

    idempotency_misses.labels(endpoint='/payments').inc()
    return await process_payment(payment)
```

3. Chaos Testing:

```
# Inject duplicate requests intentionally
class ChaosMiddleware:
    async def __call__(self, request, call_next):
        if random.random() < 0.1: # 10% chance
            # Send duplicate request
            duplicate = request.clone()
            asyncio.create_task(call_next(duplicate))

        response = await call_next(request)
        return response
```

4. Audit Logs:

```

async def audit_idempotency(key, is_duplicate, request_hash):
    await db.execute(
        """
        INSERT INTO idempotency_audit (
            key, is_duplicate, request_hash, timestamp
        ) VALUES (%s, %s, %s, NOW())
        """,
        (key, is_duplicate, request_hash)
    )

# Query for duplicates
duplicates = await db.fetchall(
    """
    SELECT key, COUNT(*) as count
    FROM idempotency_audit
    WHERE is_duplicate = true
    AND timestamp > NOW() - INTERVAL '24 hours'
    GROUP BY key
    HAVING COUNT(*) > 5 -- Suspicious activity
    """
)

```

Q20: What is the relationship between idempotency and CQRS?

Answer: CQRS (Command Query Responsibility Segregation) naturally supports idempotency.

CQRS Pattern:

```

Commands (writes):
- CreateOrder
- UpdateInventory
- ProcessPayment

Queries (reads):
- GetOrder
- GetInventory
- GetPaymentStatus

Separate models for reads and writes

```

Idempotency in CQRS:

1. Commands are naturally idempotent:

```

class CreateOrderCommand:
    def __init__(self, command_id, order_data):
        self.command_id = command_id # Idempotency key
        self.order_data = order_data

```

```

class OrderCommandHandler:
    async def handle(self, command: CreateOrderCommand):
        # Check if command already executed
        if await self.command_store.exists(command.command_id):
            # Return existing result
            return await self.command_store.get_result(command.command_id)

        # Execute command
        result = await self.create_order(command.order_data)

        # Store command execution
        await self.command_store.store(command.command_id, result)

        return result

```

2. Event Sourcing + CQRS = Natural Idempotency:

```

class Event:
    def __init__(self, event_id, event_type, data):
        self.event_id = event_id # Unique event ID
        self.event_type = event_type
        self.data = data

class EventStore:
    async def append(self, event):
        try:
            # Event ID is idempotency key
            await self.db.execute(
                """
                INSERT INTO events (event_id, event_type, data)
                VALUES (%s, %s, %s)
                """,
                (event.event_id, event.event_type, json.dumps(event.data))
            )
            return True # New event
        except UniqueViolationError:
            return False # Duplicate event (idempotent)

# Publishing events is idempotent
event = Event(
    event_id="order-created-123",
    event_type="OrderCreated",
    data={"order_id": 123}
)

await event_store.append(event) # First time: appended
await event_store.append(event) # Retry: ignored (idempotent)

```

Benefits:

- Commands have natural idempotency keys
 - Event sourcing provides audit trail
 - Read models can be rebuilt from events
 - No lost updates problem
-

Practical Implementation (Q21-Q30)

Q21: How do you implement idempotency in a REST API with Spring Boot?

Answer:

```
@RestController
@RequestMapping("/api/payments")
public class PaymentController {

    @Autowired
    private IdempotencyService idempotencyService;

    @Autowired
    private PaymentService paymentService;

    @PostMapping
    public ResponseEntity<?> createPayment(
        @RequestBody PaymentRequest request,
        @RequestHeader("Idempotency-Key") String idempotencyKey
    ) {
        // Check idempotency
        Optional<IdempotencyRecord> existing =
            idempotencyService.findByKey(idempotencyKey);

        if (existing.isPresent()) {
            IdempotencyRecord record = existing.get();

            // Verify request matches
            if (!record.matchesRequest(request)) {
                throw new IdempotencyMismatchException(
                    "Same key used for different request"
                );
            }

            // Return cached response
            return ResponseEntity
                .status(record.getResponseCode())
                .body(record.getResponseBody());
        }

        // Process payment
        try {
            PaymentResponse response = paymentService.process(request);

            // Store idempotency record
```

```

        idempotencyService.store(
            idempotencyKey,
            request,
            HttpStatus.CREATED.value(),
            response
        );

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(response);

    } catch (Exception e) {
        // Don't cache errors
        throw e;
    }
}

@Service
public class IdempotencyService {

    @Autowired
    private IdempotencyRepository repository;

    @Transactional
    public Optional<IdempotencyRecord> findByKey(String key) {
        return repository.findByKey(key)
            .filter(record -> !record.isExpired());
    }

    @Transactional
    public void store(
        String key,
        PaymentRequest request,
        int responseCode,
        PaymentResponse response
    ) {
        String requestHash = computeHash(request);

        IdempotencyRecord record = new IdempotencyRecord();
        record.setKey(key);
        record.setRequestHash(requestHash);
        record.setResponseCode(responseCode);
        record.setResponseBody(toJson(response));
        record.setExpiresAt(
            LocalDateTime.now().plusHours(24)
        );

        repository.save(record);
    }

    private String computeHash(PaymentRequest request) {
        try {
            String json = new ObjectMapper().writeValueAsString(request);

```

```

        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(json.getBytes(StandardCharsets.UTF_8));
        return Base64.getEncoder().encodeToString(hash);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}

@Entity
@Table(name = "idempotency_keys")
public class IdempotencyRecord {

    @Id
    @Column(name = "key", unique = true, nullable = false)
    private String key;

    @Column(name = "request_hash", nullable = false)
    private String requestHash;

    @Column(name = "response_code")
    private Integer responseCode;

    @Column(name = "response_body", columnDefinition = "TEXT")
    private String responseBody;

    @Column(name = "expires_at", nullable = false)
    private LocalDateTime expiresAt;

    @Column(name = "created_at")
    private LocalDateTime createdAt = LocalDateTime.now();

    public boolean isExpired() {
        return LocalDateTime.now().isAfter(expiresAt);
    }

    public boolean matchesRequest(PaymentRequest request) {
        String newHash = computeHash(request);
        return this.requestHash.equals(newHash);
    }

    // Getters and setters...
}

```

Q22: How does AWS API Gateway handle idempotency?

Answer: AWS API Gateway doesn't provide built-in idempotency. You must implement it in your Lambda functions or backend services.

Pattern: Lambda with DynamoDB

```

import boto3
import json
import hashlib
from datetime import datetime, timedelta

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('idempotency_keys')

def lambda_handler(event, context):
    # Extract idempotency key from header
    idempotency_key = event['headers'].get('Idempotency-Key')

    if not idempotency_key:
        return {
            'statusCode': 400,
            'body': json.dumps({'error': 'Idempotency-Key header required'})
        }

    # Compute request hash
    request_body = json.loads(event['body'])
    request_hash = hashlib.sha256(
        json.dumps(request_body, sort_keys=True).encode()
    ).hexdigest()

    # Check if key exists
    try:
        response = table.get_item(Key={'key': idempotency_key})

        if 'Item' in response:
            item = response['Item']

            # Verify request matches
            if item['request_hash'] != request_hash:
                return {
                    'statusCode': 422,
                    'body': json.dumps({
                        'error': 'Idempotency key reused with different request'
                    })
                }

            # Return cached response
            return {
                'statusCode': item['response_code'],
                'body': item['response_body']
            }

    except Exception as e:
        print(f"Error checking idempotency: {e}")

    # Process request (new)
    try:
        result = process_payment(request_body)

```

```

    # Store idempotency record
    ttl = int((datetime.now() + timedelta(hours=24)).timestamp())
    table.put_item(
        Item={
            'key': idempotency_key,
            'request_hash': request_hash,
            'response_code': 200,
            'response_body': json.dumps(result),
            'ttl': ttl
        }
    )

    return {
        'statusCode': 200,
        'body': json.dumps(result)
    }

except Exception as e:
    # Don't cache errors
    return {
        'statusCode': 500,
        'body': json.dumps({'error': str(e)})
    }

```

Q23: How do you handle idempotency in GraphQL mutations?

Answer: GraphQL mutations can be made idempotent by accepting an idempotency key as an argument.

Schema:

```

type Mutation {
  createPayment(
    amount: Int!
    currency: String!
    idempotencyKey: String!
  ): Payment
}

type Payment {
  id: ID!
  amount: Int!
  currency: String!
  status: PaymentStatus!
}

```

Resolver:

```

import graphene
from graphene import ObjectType, String, Int, Field, Mutation

```

```

class CreatePayment(Mutation):
    class Arguments:
        amount = Int(required=True)
        currency = String(required=True)
        idempotency_key = String(required=True)

    payment = Field(lambda: Payment)

    async def mutate(self, info, amount, currency, idempotency_key):
        # Check idempotency
        existing = await db.fetchone(
            "SELECT * FROM payments WHERE idempotency_key = %s",
            (idempotency_key,)
        )

        if existing:
            # Verify request matches
            if existing['amount'] != amount or existing['currency'] != currency:
                raise GraphQLError(
                    "Idempotency key reused with different arguments"
                )

            # Return existing payment
            return CreatePayment(payment=Payment(**existing))

        # Create new payment
        payment_id = await db.execute(
            """
            INSERT INTO payments (idempotency_key, amount, currency, status)
            VALUES (%s, %s, %s, 'completed')
            RETURNING id, amount, currency, status
            """,
            (idempotency_key, amount, currency)
        )

        payment_data = await db.fetchone(
            "SELECT * FROM payments WHERE id = %s",
            (payment_id,)
        )

        return CreatePayment(payment=Payment(**payment_data))

class Mutation(ObjectType):
    create_payment = CreatePayment.Field()

schema = graphene.Schema(mutation=Mutation)

```

Client Usage:

```

// Generate idempotency key on client
const idempotencyKey = `payment-${userId}-${Date.now()}-${uuidv4()}`;

```

```

const mutation = gql`
  mutation CreatePayment($amount: Int!, $currency: String!, $idempotencyKey:
String!) {
    createPayment(amount: $amount, currency: $currency, idempotencyKey:
$idempotencyKey) {
      id
      amount
      currency
      status
    }
  }
`;

const { data } = await client.mutate({
  mutation,
  variables: {
    amount: 10000,
    currency: "USD",
    idempotencyKey
  }
});

```

Q24: How do you implement idempotency with RabbitMQ/Kafka?

Answer: Message brokers often deliver messages multiple times. Make consumers idempotent.

RabbitMQ Consumer (Idempotent):

```

import pika
import json
import redis

redis_client = redis.Redis()

def callback(ch, method, properties, body):
    message = json.loads(body)
    message_id = message['id']

    # Check if already processed
    if redis_client.get(f"processed:{message_id}"):
        print(f"Message {message_id} already processed, skipping")
        ch.basic_ack(delivery_tag=method.delivery_tag)
        return

    try:
        # Process message
        process_order(message['data'])

        # Mark as processed (TTL: 7 days)
        redis_client.setex(f"processed:{message_id}", 604800, "1")

```

```

        # Acknowledge message
        ch.basic_ack(delivery_tag=method.delivery_tag)

    except Exception as e:
        print(f"Processing failed: {e}")
        # Negative acknowledge - message will be redelivered
        ch.basic_nack(delivery_tag=method.delivery_tag)

# Setup RabbitMQ connection
connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
channel.queue_declare(queue='orders')
channel.basic_consume(queue='orders', on_message_callback=callback)
channel.start_consuming()

```

Kafka Consumer (Idempotent):

```

from kafka import KafkaConsumer
import json
import redis

redis_client = redis.Redis()
consumer = KafkaConsumer(
    'orders',
    bootstrap_servers=['localhost:9092'],
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)

for message in consumer:
    order_data = message.value
    message_id = order_data['id']

    # Check if already processed
    if redis_client.get(f"processed:{message_id}"):
        print(f"Message {message_id} already processed")
        continue

    try:
        # Process order
        process_order(order_data)

        # Mark as processed
        redis_client.setex(f"processed:{message_id}", 604800, "1")

    except Exception as e:
        print(f"Processing failed: {e}")
        # Message will be retried

```

Kafka Producer (Idempotent):

```
from kafka import KafkaProducer

# Enable idempotent producer
producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    enable_idempotence=True, # Kafka ensures exactly-once delivery
    acks='all',
    retries=3
)

# Send message
producer.send('orders', value=order_data)
```

Q25: What database isolation levels are best for idempotency?

Answer:

| Isolation Level | Idempotency Safety | Performance | Use Case |
|------------------|--------------------|-------------|--------------------------------|
| Read Uncommitted | ✗ Unsafe | Fastest | Never for idempotency |
| Read Committed | ⚠ Partial | Fast | With application-level locking |
| Repeatable Read | ✓ Safe | Medium | Good balance |
| Serializable | ✓ Very Safe | Slowest | Critical operations |

Read Committed + Application Lock:

```
@app.post("/payments")
async def create_payment(payment: PaymentRequest, idempotency_key: str =
Header(...)):
    async with db.transaction(isolation_level='READ COMMITTED'):
        # Try to acquire lock
        result = await db.execute(
            """
            INSERT INTO idempotency_keys (key, status)
            VALUES (%s, 'processing')
            ON CONFLICT (key) DO NOTHING
            RETURNING key
            """,
            (idempotency_key,)
        )

        if result.rowcount == 0:
            # Key exists - return cached response
            return await get_cached_response(idempotency_key)

        # Process payment
        payment_result = await process_payment(payment)
```

```

# Store response
await db.execute(
    """
    UPDATE idempotency_keys
    SET status = 'completed', response = %s
    WHERE key = %s
    """,
    (json.dumps(payment_result), idempotency_key)
)

return payment_result

```

Repeatable Read (PostgreSQL Default):

```

# PostgreSQL uses Repeatable Read by default
async with db.transaction():
    # All reads within transaction see consistent snapshot
    # Safe for idempotency operations
    pass

```

Serializable (Maximum Safety):

```

async with db.transaction(isolation_level='SERIALIZABLE'):
    # Highest consistency, lowest concurrency
    # Use for critical financial operations
    pass

```

Recommendation:

- Read Committed + unique constraints: Good for most cases
- Repeatable Read: Better safety, acceptable performance
- Serializable: Only for critical operations (slow)

Q26: How do you implement idempotency in a serverless environment?

Answer: Serverless functions are stateless, so use external stores for idempotency.

AWS Lambda + DynamoDB:

```

import boto3
import json
from datetime import datetime, timedelta

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('idempotency_keys')

```

```

def lambda_handler(event, context):
    idempotency_key = event['headers'].get('Idempotency-Key')
    request_body = json.loads(event['body'])

    # Check DynamoDB for existing key
    try:
        response = table.get_item(Key={'key': idempotency_key})

        if 'Item' in response:
            # Return cached response
            return {
                'statusCode': response['Item']['status_code'],
                'body': response['Item']['response_body']
            }
    except:
        pass

    # Process request
    result = process_request(request_body)

    # Store in DynamoDB with TTL
    ttl = int((datetime.now() + timedelta(hours=24)).timestamp())
    table.put_item(
        Item={
            'key': idempotency_key,
            'status_code': 200,
            'response_body': json.dumps(result),
            'ttl': ttl
        }
    )

    return {'statusCode': 200, 'body': json.dumps(result)}

```

Azure Functions + Cosmos DB:

```

import azure.functions as func
from azure.cosmos import CosmosClient
import json

cosmos_client = CosmosClient(url, key)
database = cosmos_client.get_database_client('mydb')
container = database.get_container_client('idempotency')

def main(req: func.HttpRequest) -> func.HttpResponse:
    idempotency_key = req.headers.get('Idempotency-Key')

    # Check Cosmos DB
    try:
        item = container.read_item(
            item=idempotency_key,
            partition_key=idempotency_key

```

```

    )

    # Return cached response
    return func.HttpResponse(
        item['response_body'],
        status_code=item['status_code']
    )
except:
    pass

# Process request
result = process_request(req.get_json())

# Store in Cosmos DB
container.create_item({
    'id': idempotency_key,
    'status_code': 200,
    'response_body': json.dumps(result),
    'ttl': 86400 # 24 hours
})

return func.HttpResponse(json.dumps(result), status_code=200)

```

Google Cloud Functions + Firestore:

```

from google.cloud import firestore
import json

db = firestore.Client()

def handle_request(request):
    idempotency_key = request.headers.get('Idempotency-Key')

    # Check Firestore
    doc_ref = db.collection('idempotency_keys').document(idempotency_key)
    doc = doc_ref.get()

    if doc.exists:
        # Return cached response
        data = doc.to_dict()
        return (data['response_body'], data['status_code'])

    # Process request
    result = process_request(request.get_json())

    # Store in Firestore
    doc_ref.set({
        'status_code': 200,
        'response_body': json.dumps(result),
        'created_at': firestore.SERVER_TIMESTAMP
    })

```

```
return (json.dumps(result), 200)
```

Q27: How do you implement distributed idempotency across regions?

Answer: Multi-region idempotency requires global coordination or eventual consistency.

Pattern 1: Global Database (Strong Consistency)

Client → Regional API → Global DynamoDB (Global Tables)

↓

Replicates to all regions

Pros: Strong consistency

Cons: Higher latency, expensive

Pattern 2: Regional with Conflict Resolution

Client → Regional API → Regional DB

↓

Async replication

↓

Conflict resolution

Pros: Low latency

Cons: Eventual consistency, complex conflict resolution

Implementation (AWS DynamoDB Global Tables):

```
import boto3

# DynamoDB Global Table (replicated across regions)
dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
table = dynamodb.Table('idempotency_keys_global')

def create_payment_idempotent(payment_data, idempotency_key):
    try:
        # Try to create item (conditional write)
        table.put_item(
            Item={
                'idempotency_key': idempotency_key,
                'region': 'us-east-1',
                'payment_data': payment_data,
                'status': 'processing',
                'ttl': get_ttl(24) # 24 hours
            },
```

```

        ConditionExpression='attribute_not_exists(idempotency_key)'
    )

    # Process payment
    result = process_payment(payment_data)

    # Update status
    table.update_item(
        Key={'idempotency_key': idempotency_key},
        UpdateExpression='SET #status = :status, result = :result',
        ExpressionAttributeNames={'#status': 'status'},
        ExpressionAttributeValues={
            ':status': 'completed',
            ':result': result
        }
    )

    return result

except ClientError as e:
    if e.response['Error']['Code'] == 'ConditionalCheckFailedException':
        # Key exists - fetch result
        response = table.get_item(Key={'idempotency_key': idempotency_key})
        return response['Item']['result']
    raise

```

Pattern 3: Regional Locks + Eventual Consistency

```

# Each region has its own Redis
regional_redis = {
    'us-east-1': redis.Redis(host='redis-us-east-1'),
    'eu-west-1': redis.Redis(host='redis-eu-west-1'),
    'ap-south-1': redis.Redis(host='redis-ap-south-1')
}

def process_with_regional_idempotency(region, idempotency_key, operation):
    redis_client = regional_redis[region]

    # Check regional cache
    cached = redis_client.get(f"idem:{idempotency_key}")
    if cached:
        return json.loads(cached)

    # Process operation
    result = operation()

    # Cache in regional Redis
    redis_client.setex(f"idem:{idempotency_key}", 86400, json.dumps(result))

    # Replicate to central store (async)
    asyncio.create_task(replicate_to_global_store(idempotency_key, result))

```

```
return result
```

Q28: How do you audit idempotency violations?

Answer: Implement logging and monitoring to detect idempotency violations.

Audit Table:

```
CREATE TABLE idempotency_audit (  
    id SERIAL PRIMARY KEY,  
    idempotency_key VARCHAR(255) NOT NULL,  
    request_hash VARCHAR(64) NOT NULL,  
    response_hash VARCHAR(64),  
    is_duplicate BOOLEAN,  
    endpoint VARCHAR(255),  
    user_id INT,  
    ip_address VARCHAR(45),  
    created_at TIMESTAMP DEFAULT NOW(),  
    INDEX idx_key (idempotency_key),  
    INDEX idx_created_at (created_at)  
);
```

Logging:

```
async def audit_idempotency_check(  
    idempotency_key: str,  
    request_hash: str,  
    is_duplicate: bool,  
    endpoint: str,  
    user_id: int,  
    ip_address: str  
):  
    await db.execute(  
        """  
        INSERT INTO idempotency_audit (  
            idempotency_key,  
            request_hash,  
            is_duplicate,  
            endpoint,  
            user_id,  
            ip_address  
        ) VALUES (%s, %s, %s, %s, %s, %s)  
        """,  
        (idempotency_key, request_hash, is_duplicate, endpoint, user_id,  
        ip_address)  
    )  
  
    @app.post("/payments")
```

```

async def create_payment(
    payment: PaymentRequest,
    idempotency_key: str = Header(...),
    request: Request
):
    request_hash = compute_hash(payment.dict())
    is_duplicate = False

    # Check idempotency
    existing = await check_idempotency(idempotency_key)
    if existing:
        is_duplicate = True

    # Verify request matches
    if existing['request_hash'] != request_hash:
        # VIOLATION DETECTED!
        await audit_idempotency_check(
            idempotency_key,
            request_hash,
            is_duplicate,
            '/payments',
            payment.user_id,
            request.client.host
        )

        # Alert
        await send_alert(
            f"Idempotency violation: key={idempotency_key}, "
            f"user={payment.user_id}, ip={request.client.host}"
        )

        raise HTTPException(422, "Idempotency key reused with different
request")

    # Audit successful check
    await audit_idempotency_check(
        idempotency_key,
        request_hash,
        is_duplicate,
        '/payments',
        payment.user_id,
        request.client.host
    )

    # Process payment...

```

Monitoring Query:

```

-- Find suspicious activity (many retries)
SELECT
    idempotency_key,
    COUNT(*) as retry_count,

```

```

COUNT(DISTINCT request_hash) as unique_requests,
MIN(created_at) as first_attempt,
MAX(created_at) as last_attempt
FROM idempotency_audit
WHERE created_at > NOW() - INTERVAL '1 hour'
GROUP BY idempotency_key
HAVING COUNT(*) > 10 -- More than 10 retries
OR COUNT(DISTINCT request_hash) > 1 -- Different request bodies
ORDER BY retry_count DESC;

```

Q29: How do you implement idempotency for batch operations?

Answer: Batch operations need idempotency at both batch and item levels.

Pattern 1: Batch-Level Idempotency

```

@app.post("/batch/payments")
async def create_batch_payments(
    batch: BatchPaymentRequest,
    batch_idempotency_key: str = Header(...)
):
    # Check if batch already processed
    existing_batch = await db.fetchone(
        "SELECT * FROM batch_results WHERE batch_key = %s",
        (batch_idempotency_key,)
    )

    if existing_batch:
        return existing_batch['results']

    # Process batch
    results = []
    for payment in batch.payments:
        # Each item has its own key
        item_key = f"{batch_idempotency_key}:item:{payment.id}"
        result = await create_payment_idempotent(payment, item_key)
        results.append(result)

    # Store batch result
    await db.execute(
        """
        INSERT INTO batch_results (batch_key, results, created_at)
        VALUES (%s, %s, NOW())
        """,
        (batch_idempotency_key, json.dumps(results))
    )

    return results

```

Pattern 2: Item-Level Idempotency (Partial Retry)

```

@app.post("/batch/payments")
async def create_batch_payments(
    batch: BatchPaymentRequest,
    batch_idempotency_key: str = Header(...)
):
    results = []

    for payment in batch.payments:
        # Each item has unique key
        item_key = f"{batch_idempotency_key}:item:{payment.id}"

        try:
            result = await create_payment_idempotent(payment, item_key)
            results.append({
                'id': payment.id,
                'status': 'success',
                'result': result
            })
        except Exception as e:
            results.append({
                'id': payment.id,
                'status': 'failed',
                'error': str(e)
            })

    return {
        'batch_id': batch_idempotency_key,
        'results': results,
        'summary': {
            'total': len(batch.payments),
            'succeeded': sum(1 for r in results if r['status'] == 'success'),
            'failed': sum(1 for r in results if r['status'] == 'failed')
        }
    }

```

Pattern 3: Cursor-Based Batch Processing

```

@app.post("/batch/payments")
async def create_batch_payments(
    batch: BatchPaymentRequest,
    batch_idempotency_key: str = Header(...)
):
    # Get or create batch state
    batch_state = await get_or_create_batch_state(batch_idempotency_key)

    # Process from last cursor
    cursor = batch_state.cursor
    results = batch_state.results

    for i in range(cursor, len(batch.payments)):

```

```

        payment = batch.payments[i]
        item_key = f"{batch_idempotency_key}:item:{payment.id}"

        try:
            result = await create_payment_idempotent(payment, item_key)
            results.append(result)

            # Update cursor (checkpoint)
            await update_batch_cursor(batch_idempotency_key, i + 1, results)

        except Exception as e:
            # Retry from current cursor
            raise

    # Mark batch as complete
    await complete_batch(batch_idempotency_key)

    return results

```

Q30: How do you implement idempotency with webhooks?

Answer: Webhooks are often retried by providers. Make webhook handlers idempotent.

Webhook Handler (Idempotent):

```

@app.post("/webhooks/stripe")
async def handle_stripe_webhook(request: Request):
    payload = await request.body()
    sig_header = request.headers.get('Stripe-Signature')

    # Verify webhook signature
    try:
        event = stripe.Webhook.construct_event(
            payload, sig_header, webhook_secret
        )
    except ValueError:
        raise HTTPException(400, "Invalid payload")
    except stripe.error.SignatureVerificationError:
        raise HTTPException(400, "Invalid signature")

    # Use event ID as idempotency key
    event_id = event['id'] # e.g., "evt_1a2b3c4d"

    # Check if already processed
    processed = await redis.get(f"webhook:processed:{event_id}")
    if processed:
        logger.info(f"Webhook {event_id} already processed")
        return {"status": "ok", "message": "already_processed"}

    try:
        # Process webhook based on type

```

```

        if event['type'] == 'payment_intent.succeeded':
            await handle_payment_success(event['data']['object'])
        elif event['type'] == 'payment_intent.failed':
            await handle_payment_failure(event['data']['object'])

        # Mark as processed (TTL: 7 days)
        await redis.setex(f"webhook:processed:{event_id}", 604800, "1")

        return {"status": "ok"}

    except Exception as e:
        logger.error(f"Webhook processing failed: {e}")
        # Don't mark as processed - allow retry
        raise HTTPException(500, "Processing failed")

```

Database-Based Webhook Deduplication:

```

@app.post("/webhooks/payment-provider")
async def handle_webhook(webhook: WebhookPayload):
    webhook_id = webhook.id

    try:
        # Try to insert webhook record
        await db.execute(
            """
            INSERT INTO webhook_events (
                webhook_id,
                event_type,
                payload,
                status,
                created_at
            ) VALUES (%s, %s, %s, 'processing', NOW())
            """,
            (webhook_id, webhook.type, json.dumps(webhook.data))
        )
    except UniqueViolationError:
        # Already processed
        logger.info(f"Webhook {webhook_id} already processed")
        return {"status": "ok", "message": "duplicate"}

    try:
        # Process webhook
        await process_webhook(webhook)

        # Update status
        await db.execute(
            """
            UPDATE webhook_events
            SET status = 'completed', processed_at = NOW()
            WHERE webhook_id = %s
            """,
            (webhook_id,)
        )
    
```

```
)

    return {"status": "ok"}

except Exception as e:
    # Mark as failed (allow retry)
    await db.execute(
        """
        UPDATE webhook_events
        SET status = 'failed', error = %s
        WHERE webhook_id = %s
        """,
        (str(e), webhook_id)
    )
    raise

# Schema
CREATE TABLE webhook_events (
    webhook_id VARCHAR(255) PRIMARY KEY,
    event_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    status VARCHAR(50) NOT NULL,
    error TEXT,
    created_at TIMESTAMP DEFAULT NOW(),
    processed_at TIMESTAMP
);
```
